Programming in Python and Deep Learning

Forecasting with Machine Learning

Natasia Villadsen

2026-11-04

Introduction

Today

Brains and history From biological neurons to the renaissance of neural networks

Anatomy of a network Layers, activations, and the forecast

Training Objective, optimization, backpropagation

Regularization Dropout, penalties, and early stopping

Implementation The PyTorch pipeline, from data to evaluation

Beyond feed-forward neural networks CNNs, LSTMs, and Transformers

Why PyTorch?

PyTorch is the deep learning framework of this course

40.5% vs. 34.7%	248,163 vs. 17,342	86.2M vs. 16.7M
Job postings (PyTorch vs. TF)	Hugging Face models (PyTorch vs. TF)	PyPI downloads (PyTorch vs. TF), (Aug 2026)

Sources: Huntr (Jul 2026); Second Talent (2026)

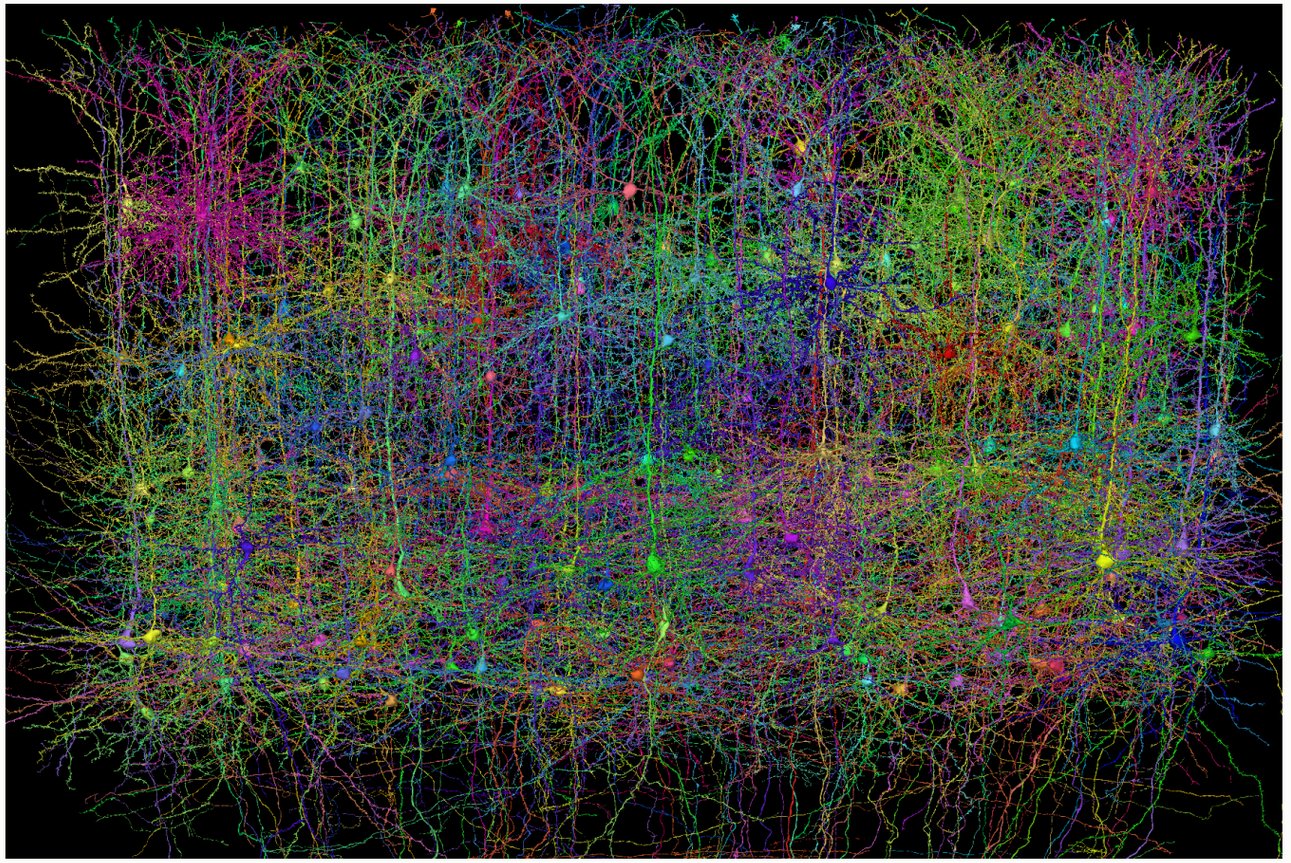
- **Tutorials**

> PyTorch is the framework of instruction, but you are free to use TensorFlow

Neural Networks are Inspired by Brains

A Neural Network of the Brain of a Mouse I

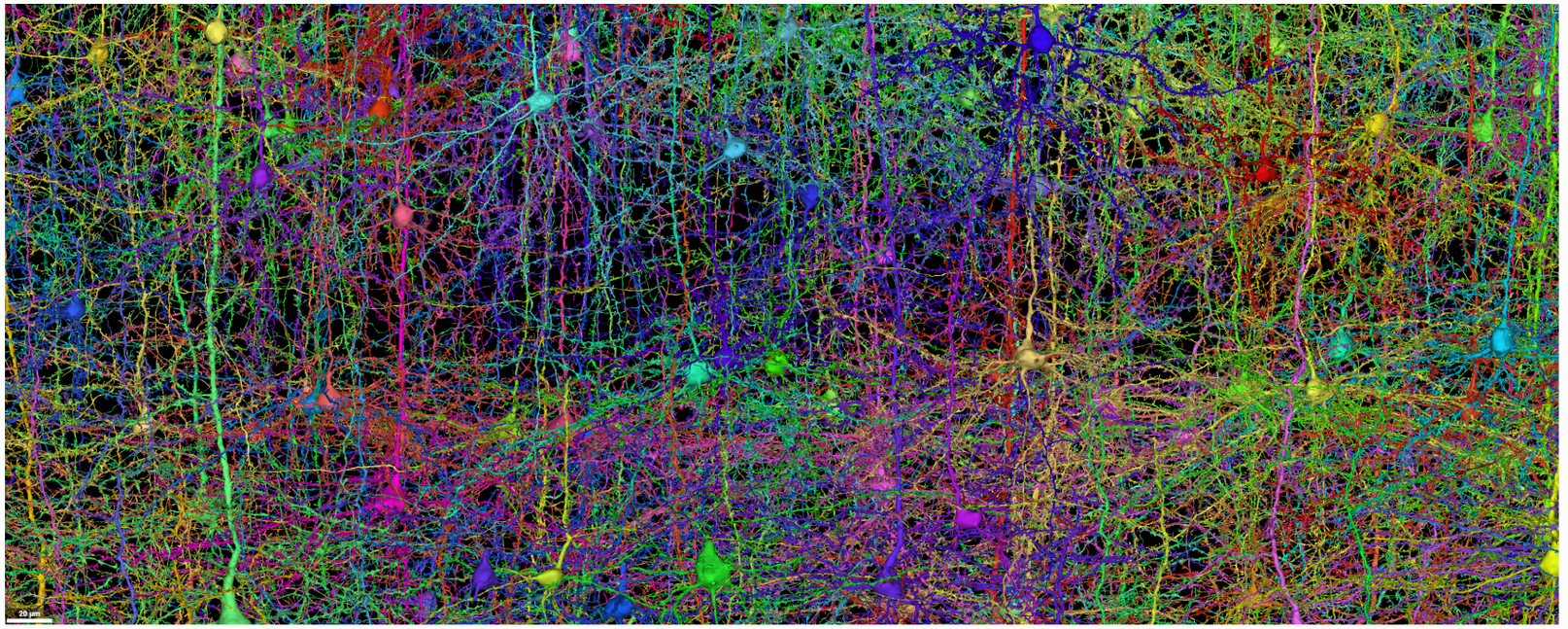
Extract from 1 mm^3 of Mouse Brain



Source: MICrONS Explorer Render, Daniel Berger & MICrONS Explorer Consortium Team. Google Research and Harvard University.

A Neural Network of the Brain of a Mouse II

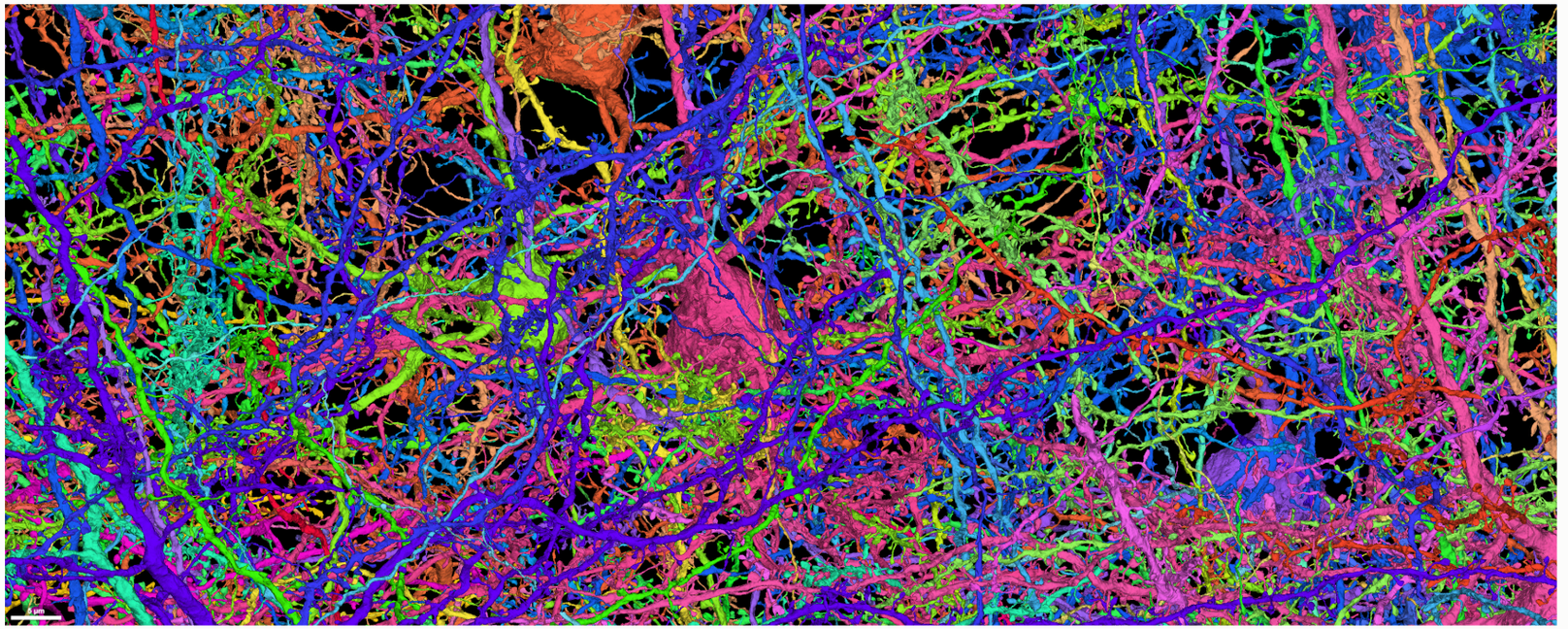
Extract from 1 mm³ of Mouse Brain — Closer Look



Source: MICrONS Explorer Render, Daniel Berger & MICrONS Explorer Consortium Team. Google Research and Harvard University.

A Neural Network of the Brain of a Mouse III

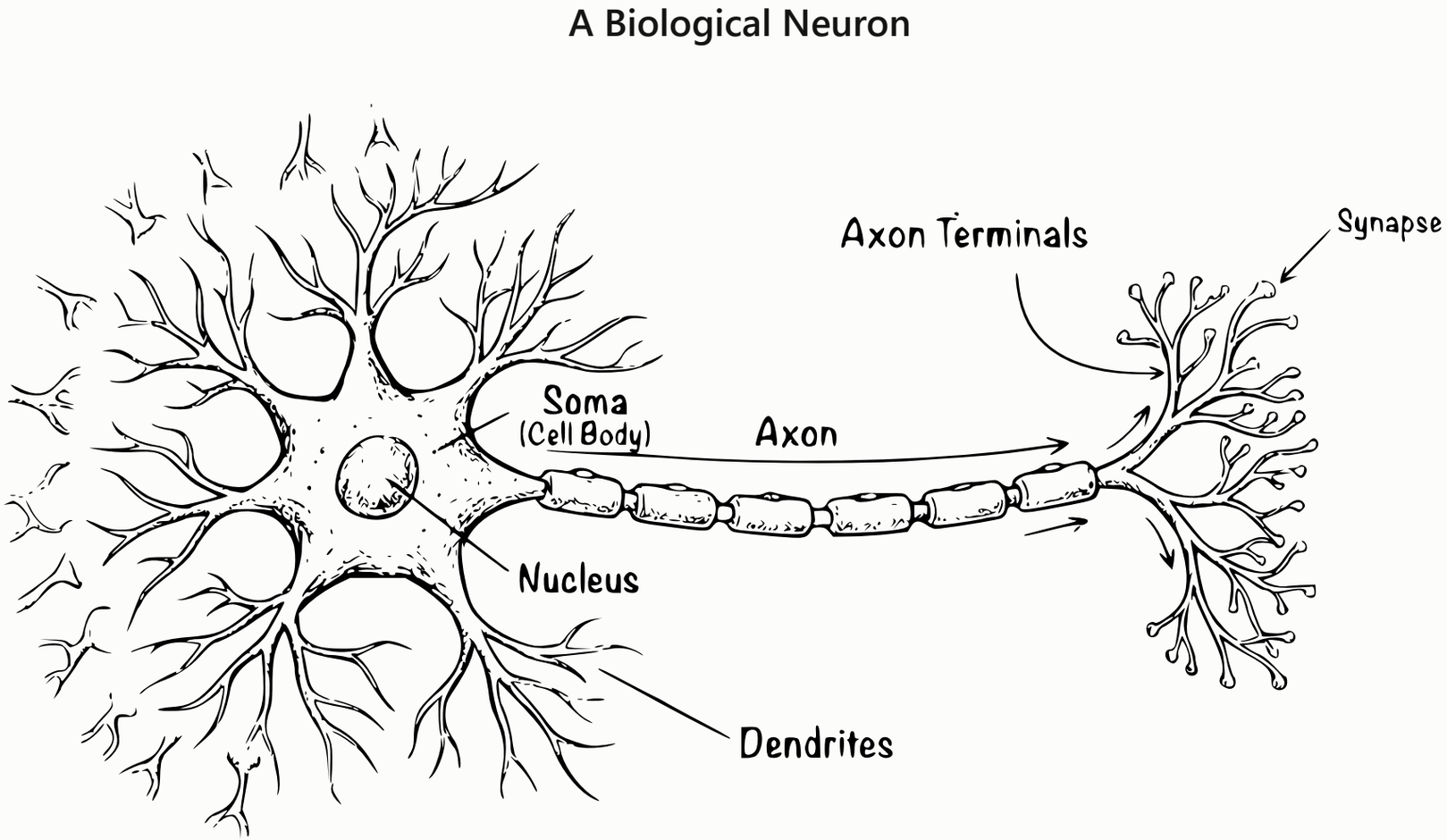
Extract from 1 mm^3 of Mouse Brain — Even Closer Look



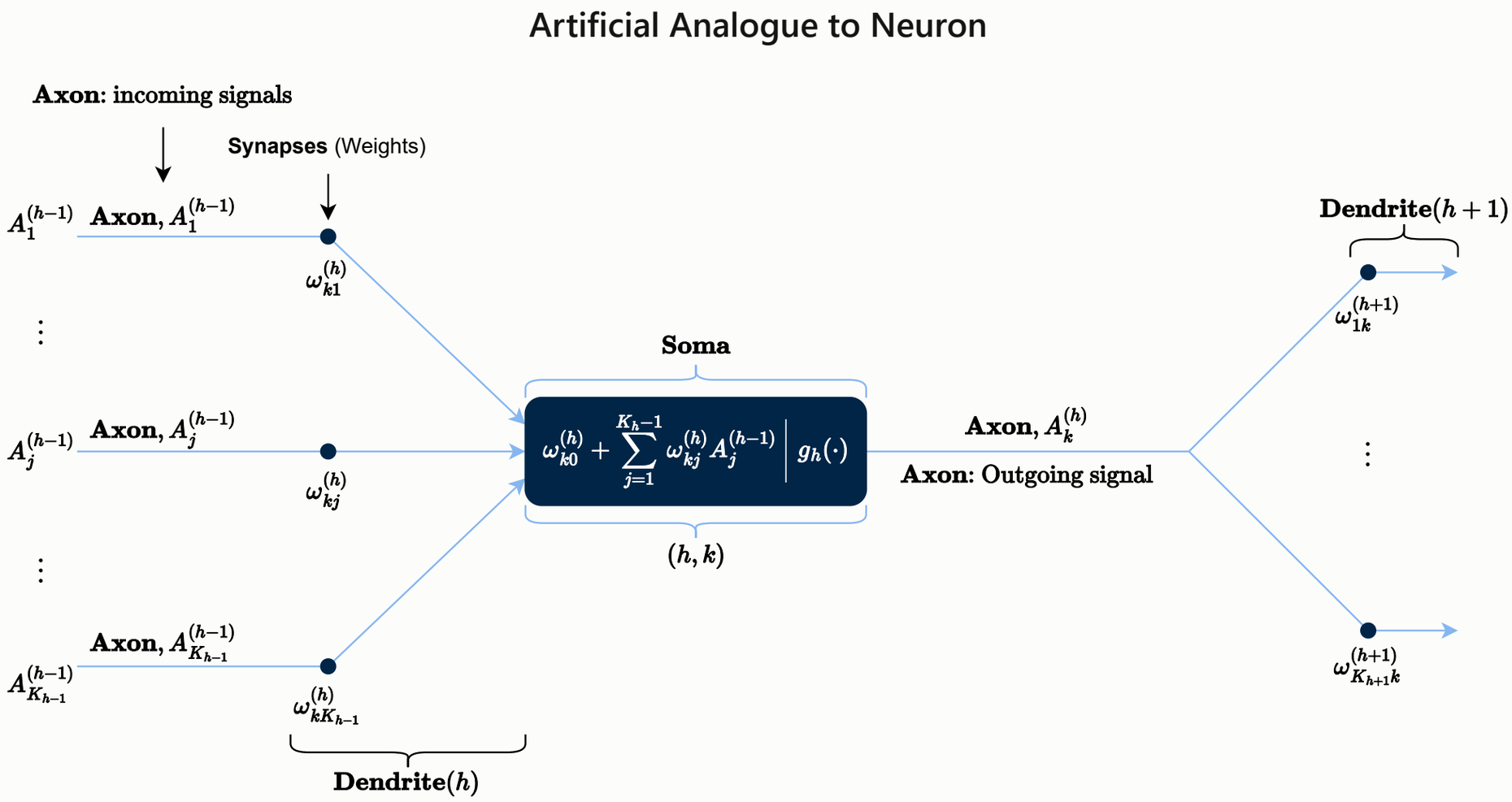
Source: MICrONS Explorer Render, Daniel Berger & MICrONS Explorer Consortium Team. Google Research and Harvard University.

Explore the network yourself.

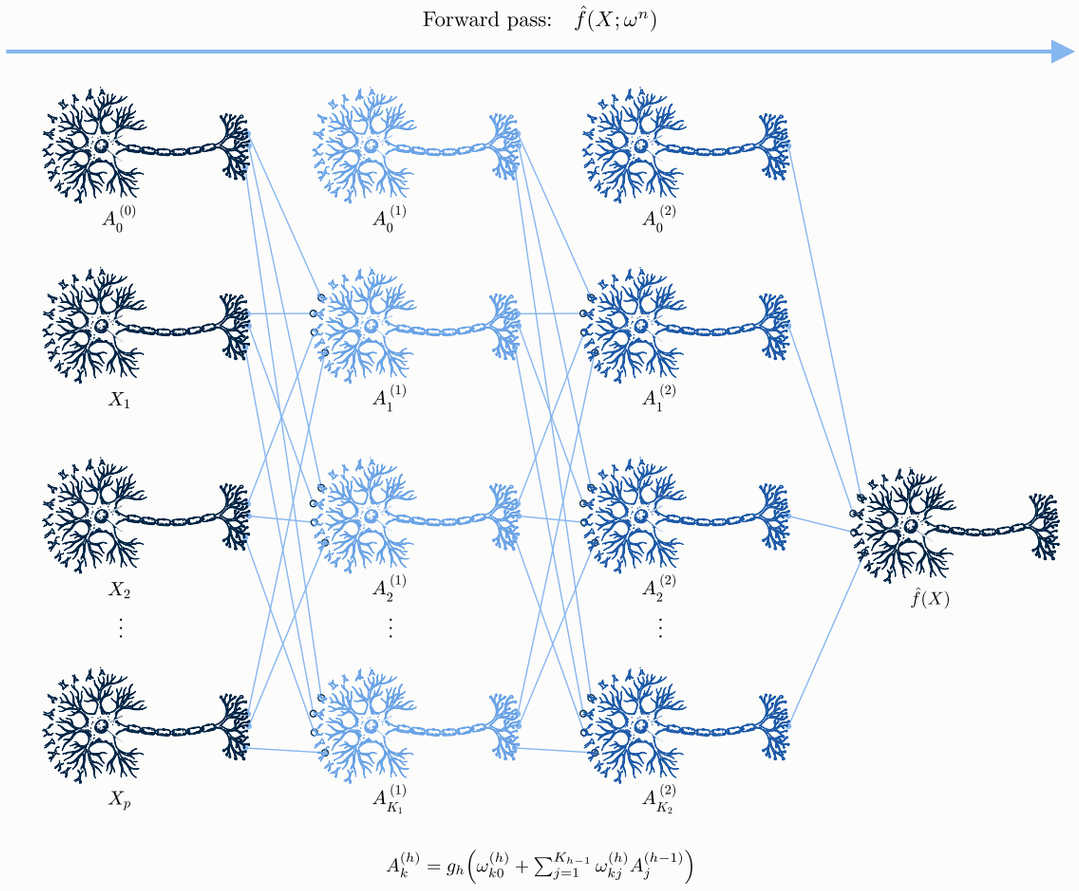
A Neuron I


A Biological Neuron
Axon Terminals
Synapse
Soma
(Cell Body)
Axon
Nucleus
Dendrites

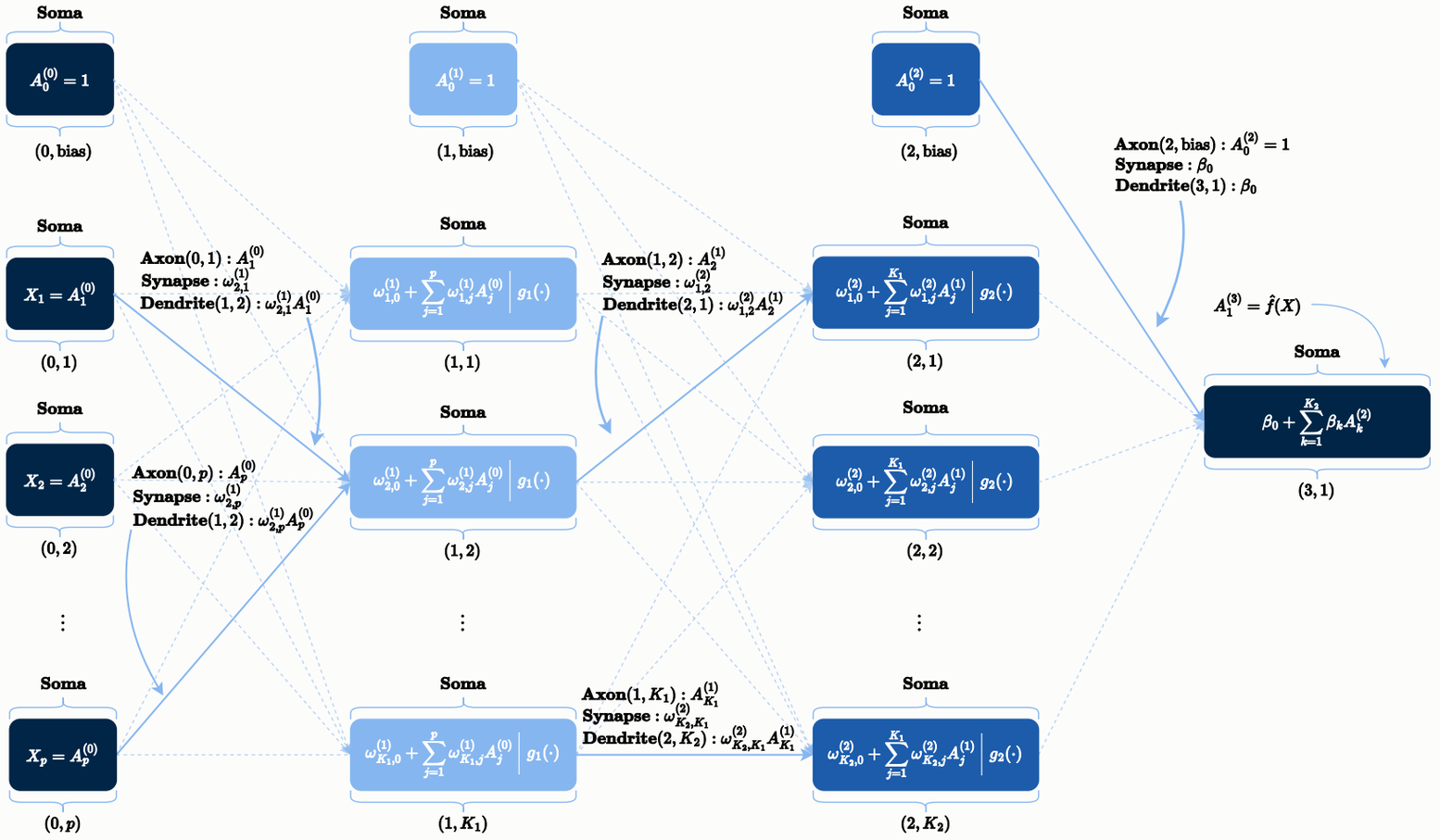
A Neuron II


Artificial Analogue to Neuron
Axon: incoming signals
Synapses (Weights)
$A_1^{(h-1)}$ Axon, $A_1^{(h-1)}$
$\omega_{k1}^{(h)}$
$A_j^{(h-1)}$ Axon, $A_j^{(h-1)}$
$\omega_{kj}^{(h)}$
$A_{K_{h-1}}^{(h-1)}$ Axon, $A_{K_{h-1}}^{(h-1)}$
$\omega_{kK_{h-1}}^{(h)}$
Dendrite(h)
Soma
$\omega_{k0}^{(h)} + \sum_{j=1}^{K_h-1} \omega_{kj}^{(h)} A_j^{(h-1)} \mid g_h(\cdot)$
(h, k)
Axon, $A_k^{(h)}$
Axon: Outgoing signal
Dendrite$(h+1)$
$\omega_{1k}^{(h+1)}$
$\omega_{K_{h+1}k}^{(h+1)}$

The Neurological Network



The Artificial Network



Neural Networks are Supported by...

Computational Power

Exponentially growing computing power, GPUs

Backpropagation

Efficient gradient computation for deep networks

Pre-trained Models

Including custom tuning

Frameworks

PyTorch, TensorFlow, and similar

Next challenge: Physical limits of silicon-based chips reached.
Possible solution: quantum computing

Neural Networks

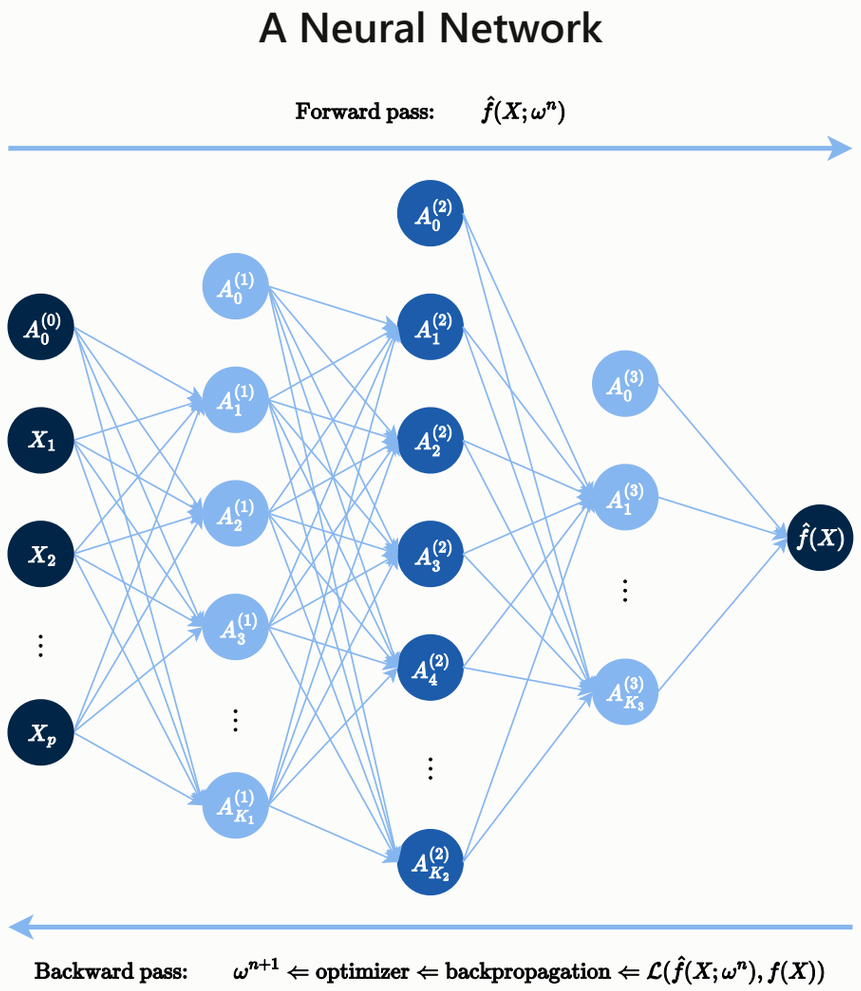
Forecasting US Inflation: Benchmarks vs. a Neural Network

US quarterly inflation, test sample (last 15% of quarters)

FORECAST	TEST MSE
Random walk	13.8971
OLS	9.9284
LASSO	9.5005
Ridge	9.5995
Random forest	9.9475
Feedforward neural network - basic	10.8601

> Today: how do we construct and train this network?

A Neural Network



The Layers of a Neural Network

Layers are indexed by h, $h = 0, \ldots, H+1$. There are K_h **neurons** in each layer h, with **activations** $A_k^{(h)}$

ONE INPUT LAYER

$h = 0$

The neurons are the p input features, $\mathbf{X} = (X_1, X_2, \ldots, X_p)$

$A_j^{(0)} = X_j, \quad j = 1, \ldots, p$

H HIDDEN LAYERS

$h = 1, \ldots, H$

Essential computations here

ONE OUTPUT LAYER

$h = H + 1$

Produces the forecast

The Hidden Layers

The essential computations occur in the **hidden** layers ($h = 1, \ldots, H$), with activations $A_k^{(h)}$, weights $\omega_{kj}^{(h)}$, biases $\omega_{k0}^{(h)}$, and activation functions g_h,

$$A_k^{(h)} = g_h\left(\omega_{k0}^{(h)} + \sum_{j=1}^{K_{h-1}} \omega_{kj}^{(h)} A_j^{(h-1)}\right), \qquad k = 1, \ldots, K_h$$

WEIGHTED SUM

Weights times the activations of the previous layer

ACTIVATION FUNCTION

g_h introduces non-linearity to layer h

The First Hidden Layer

The first hidden layer is a special case, with $K_0 = p$ and $A_j^{(0)} = X_j$,

$$A_k^{(1)} = g_1\left(\omega_{k0}^{(1)} + \sum_{j=1}^{p} \omega_{kj}^{(1)} X_j\right), \qquad k = 1, \ldots, K_1$$

Forecasts

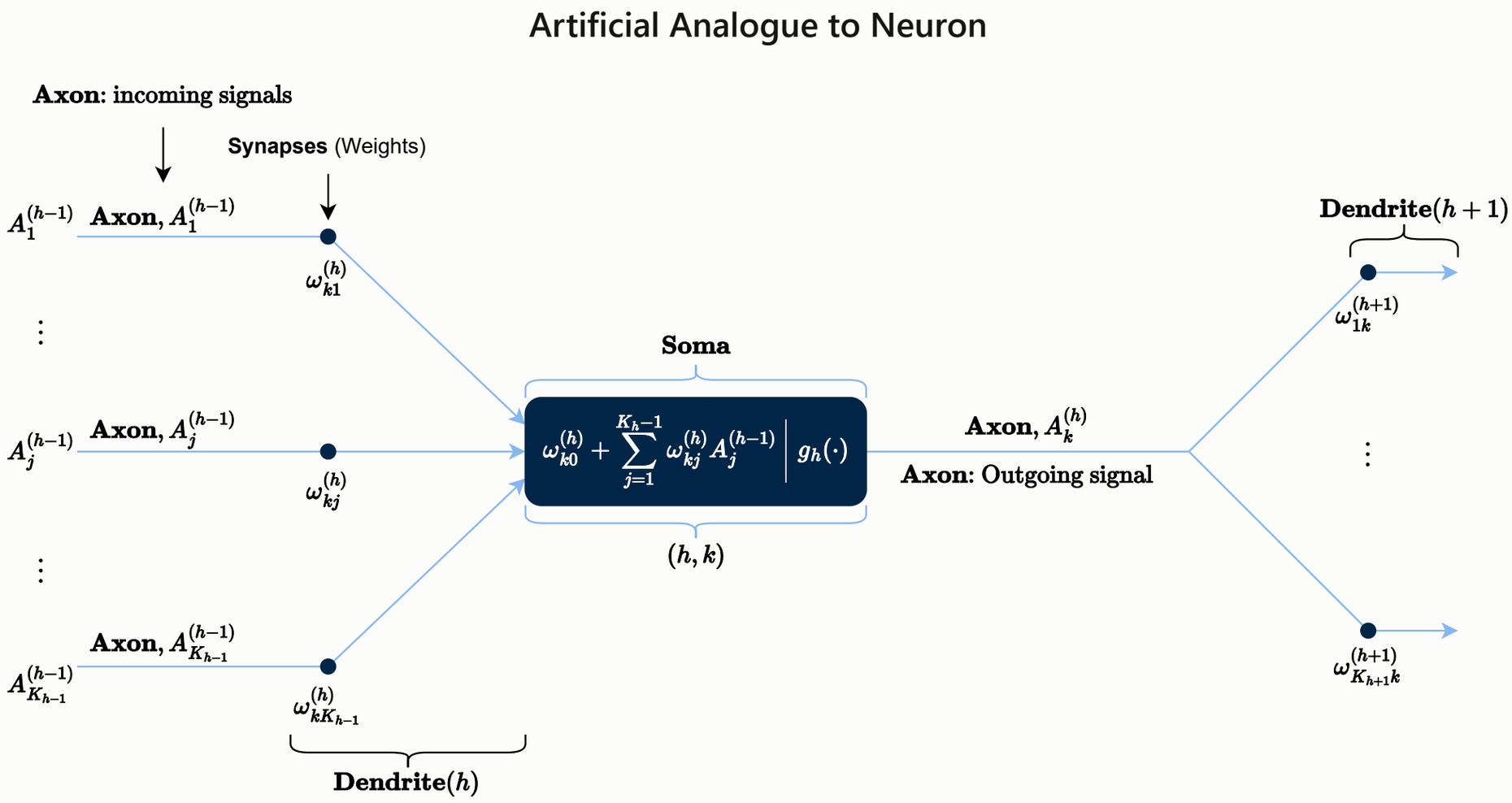
In the regression task, the **output** layer ($h = H + 1$) produces forecasts, with weights β_k and intercept β_0,

$$f(\mathbf{X}) = \beta_0 + \sum_{k=1}^{K_H} \beta_k A_k^{(H)}$$

Evaluating in the activations of the last hidden layer,

$$f(\mathbf{X}) = \beta_0 + \sum_{k=1}^{K_H} \beta_k \, g_H \left(\omega_{k0}^{(H)} + \sum_{j=1}^{K_{H-1}} \omega_{kj}^{(H)} A_j^{(H-1)} \right)$$

Similarity to Neurons Revisited



Implementation

The Blueprint

1	**Data**	Target variable and predictors
2	**Split and scale**	Chronological training, validation and test split, standardization fitted on the training sample only
3	**DataLoaders**	Tensors in mini-batches
4	**Model**	An `nn.Module` defining the architecture
5	**Ingredients**	Objective, optimizer, regularization, loss
6	**Training loop**	Training and validation over epochs
7	**Evaluation**	Out-of-sample accuracy measures on the test sample

Preliminaries

Importing libraries and selecting device

Python Code | Start Over | Run Code

```
import copy
import time
import numpy as np
import statsmodels.api as sm
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import train_test_split

# import torch
# import torch.nn as nn
# import torch.nn.functional as F
# from torch.utils.data import TensorDataset, DataLoader

# device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
```

Empirical Data

US quarterly macro data 1959Q1-2009Q3 (no missing values).

Python Code | Start Over | Run Code

```
df = sm.datasets.macrodata.load_pandas().data.iloc[1:] # No inflation in the first quarter
features = df[["infl", "unemp", "tbilrate"]].to_numpy()   # (T, 3)
infl = features[:, 0]
```

Lagging features: inflation, unemployment and the T-bill rate at time t are used to predict inflation at time $t + 1$,

Python Code | Start Over | Run Code

```
X = features[:-1]
target = infl[1:].reshape(-1, 1)
```

Temporally Conscious Data Splitting

Chronological split (shuffle=False) to preserve temporal order (split: 70:15:15)

Python Code

Start Over

Run Code

```python
X_train, X_tmp, y_train, y_tmp = train_test_split(X, target, train_size=0.7, shuffle=False)
X_val, X_test, y_val, y_test = train_test_split(X_tmp, y_tmp, train_size=0.5, shuffle=False)

print(f"Train: {X_train.shape}, Validation: {X_val.shape}, Test: {X_test.shape}")
```

```
Train: (140, 3), Validation: (30, 3), Test: (31, 3)
```

Standardization

Standardization, fitting on the training sample only,

Python Code | Start Over | Run Code

```
X_scaler = StandardScaler()
y_scaler = StandardScaler()

X_train = X_scaler.fit_transform(X_train)
X_val = X_scaler.transform(X_val)
X_test = X_scaler.transform(X_test)

y_train = y_scaler.fit_transform(y_train)
y_val = y_scaler.transform(y_val)
y_test = y_scaler.transform(y_test)
```

Tensors

NumPy array → `torch.Tensor` → `TensorDataset` → `DataLoader`

The input data is converted to the format that PyTorch expects/receives,

Python Code

```
X_train = torch.from_numpy(X_train).float().to(device)
X_val = torch.from_numpy(X_val).float().to(device)
X_test = torch.from_numpy(X_test).float().to(device)
y_train = torch.from_numpy(y_train).float().to(device)
y_val = torch.from_numpy(y_val).float().to(device)
y_test = torch.from_numpy(y_test).float().to(device)
```

Constructing DataLoaders

This is incredibly efficient for large or specialized datasets,

Python Code

```python
dfTrain = TensorDataset(X_train, y_train)
dfVal = TensorDataset(X_val, y_val)
dfTest = TensorDataset(X_test, y_test)

lTrain = DataLoader(dfTrain, batch_size=32, shuffle=True)
lVal = DataLoader(dfVal, batch_size=32, shuffle=False)
lTest = DataLoader(dfTest, batch_size=32, shuffle=False)
```

The Simple Neural Network I

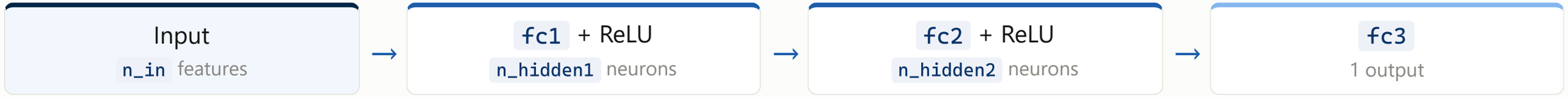
Python Code

```python
class NN(nn.Module):
    def __init__(self, n_in, n_hidden1=16, n_hidden2=8):
        super(NN, self).__init__()
        self.fc1 = nn.Linear(n_in, n_hidden1)
        self.fc2 = nn.Linear(n_hidden1, n_hidden2)
        self.fc3 = nn.Linear(n_hidden2, 1)

    def forward(self, x):
        x = F.relu(self.fc1(x))
        x = F.relu(self.fc2(x))
        x = self.fc3(x)

        return x
```

Parameters are passed after `self` (here `n_in`, `n_hidden1` and `n_hidden2`)

The Simple Neural Network II

PyTorch neural networks are defined as subclasses of `nn.Module`



- `n_in` is the number of input features, `n_hidden1` and `n_hidden2` are the numbers of neurons in the first and second hidden layer
- The network on the previous slide has two hidden layers

Non-Linear Activation Functions

Introduce non-linearity, allowing the model to learn complex patterns

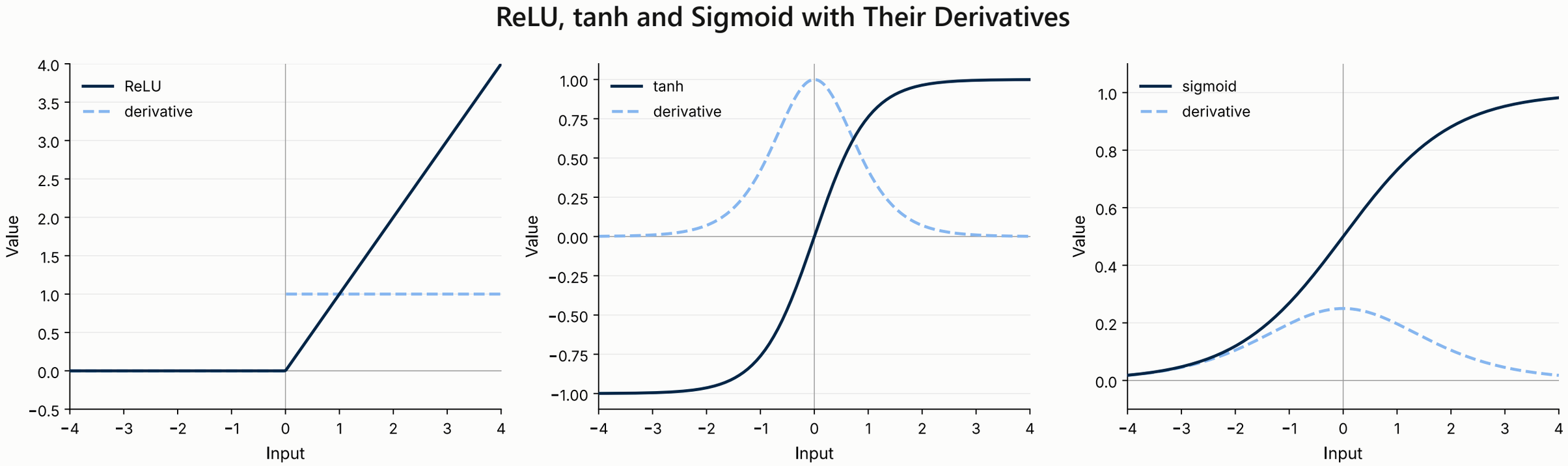
Function	Map	Definition
ReLU (Rectified Linear Unit) most prevalent today	$f : \mathbb{R} \to [0, \infty)$	$f(x) = \max\{0, x\}$
tanh (hyperbolic tangent)	$\tanh : \mathbb{R} \to (-1, 1)$	$\tanh(x) = \dfrac{e^x - e^{-x}}{e^x + e^{-x}}$
sigmoid	$\sigma : \mathbb{R} \to (0, 1)$	$\sigma(x) = \dfrac{1}{1 + e^{-x}}$

SIGMOID: THE FIRST ACTIVATION FUNCTION

$\sigma(x) \approx 0$: the neuron does not fire

$\sigma(x) \approx 1$: the neuron fires $\Rightarrow$ the neuron contributes to the output of the network

Activation Functions and Their Derivatives



ReLU: $g'(x) = 0$ for $x < 0$, $g'(x) = 1$ for $x > 0$. tanh: $g'(x) = 1 - \tanh^2(x)$. sigmoid: $g'(x) = \sigma(x)(1 - \sigma(x))$.

The derivative g' enters every backpropagation step (coming up).

Note: the sigmoid derivative is maximized at $x = 0 \rightarrow \sigma'(x) = 0.25$ – risk of vanishing gradients

Specifying the Activation Function

Specifying the activation function in the forward pass,

Python Code

```
x = F.relu(self.fc1(x))
```

Alternatives: `torch.tanh(self.fc1(x))`, `torch.sigmoid(self.fc1(x))`

Up to you: Activation Functions

The vanishing gradient problem pertains to the maximum of the sigmoid derivative being 0.25. A product of many such derivatives will with high likelihood approach zero.

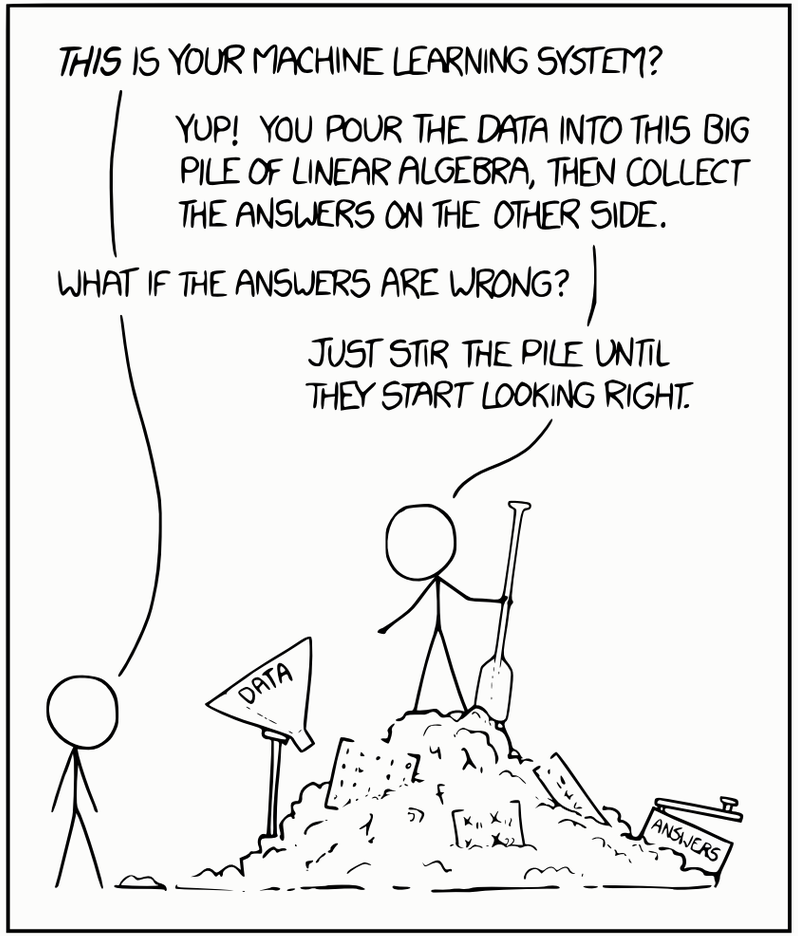
Using the derivative functions of the ReLU and sigmoid activation functions, `d_sigmoid` and `d_relu`, compute the product of each evaluated at $x = 1$, ten times (the vector `np.full(10, 1.0)`).

Exercise | Start Over | Show Solution | Run Code

```
x = np.full(10, 1.0)
prod_sig  = ...
prod_relu = ...
print(f"Sigmoid: {prod_sig:.1e} | ReLU: {prod_relu:.1e}")
```

Training a Neural Network I

xkcd #1838



Source: Randall Munroe, https://xkcd.com/1838/, CC BY-NC 2.5.

Training a Neural Network II

We give this example in the $h = 1$ case,

$$f(\mathbf{X}) = \beta_0 + \sum_{k=1}^{K_1} \beta_k A_k^{(1)} = \beta_0 + \sum_{k=1}^{K_1} \beta_k g_1 \left(\omega_{k0}^{(1)} + \sum_{j=1}^{p} \omega_{kj}^{(1)} X_j \right)$$

The parameters to be estimated are,

OUTPUT LAYER

$\boldsymbol{\beta} = \beta_0, \beta_1, \ldots, \beta_{K_1}$

HIDDEN LAYER

$\boldsymbol{\omega}^{(1)} = \omega_{kj}^{(1)}$ for $k = 1, \ldots, K_1$ and $j = 0, \ldots, p$

Stack the parameters into a vector: $\theta = (\boldsymbol{\beta}, \boldsymbol{\omega}^{(1)})$

Objective Function

We assume a squared loss function to estimate parameters θ.

For a single-hidden-layer network ($H = 1$), the model forecast for observation i is,

$$f_\theta(x_i) = \beta_0 + \sum_{k=1}^{K_1} \beta_k g_1 \left(\omega_{k0}^{(1)} + \sum_{j=1}^{p} \omega_{kj}^{(1)} x_{ij} \right)$$

Vectorising the parameters into θ, we minimize the objective function $R(\theta)$:

$$\min_\theta \{R(\theta)\}, \quad R(\theta) = \frac{1}{2} \sum_{i=1}^{n} (y_i - f_\theta(x_i))^2$$

Numerical Optimization I

Non-convex and high-dimensional problem
no closed form, no analytical solution

→

Numerical optimization
gradient descent is a common algorithm

→

Too heavy computationally
stochastic gradient descent (SGD) is used instead

SGD uses only a small batch of data to compute the gradient

Numerical Optimization II — Gradient Descent

- Given a differentiable function $f : \mathbb{R}^p \to \mathbb{R}$ to be minimized with respect to the vector $\theta \in \mathbb{R}^p$ and step size s,

1. Initialize with some starting value $\theta^*_{(0)}$
2. Then, for each iteration, i,
    - Compute the gradient at $\theta^*_{(i)}$, i.e.,

$$\nabla f(\theta^*_{(i)}) = \left.\frac{\partial f(\theta)}{\partial \theta}\right|_{\theta=\theta^*_{(i)}}$$

    - Set $\theta^*_{(i+1)} = \theta^*_{(i)} - s\nabla f(\theta^*_{(i)})$
3. Repeat until convergence. A convergence criterion could be a sufficiently small gradient, $\|\nabla f(\theta^*_{(i)})\| < \varepsilon$ with a maximal number of iterations at which the algorithm terminates – and reports non-convergence.

Numerical Optimization III — Stochastic Gradient Descent

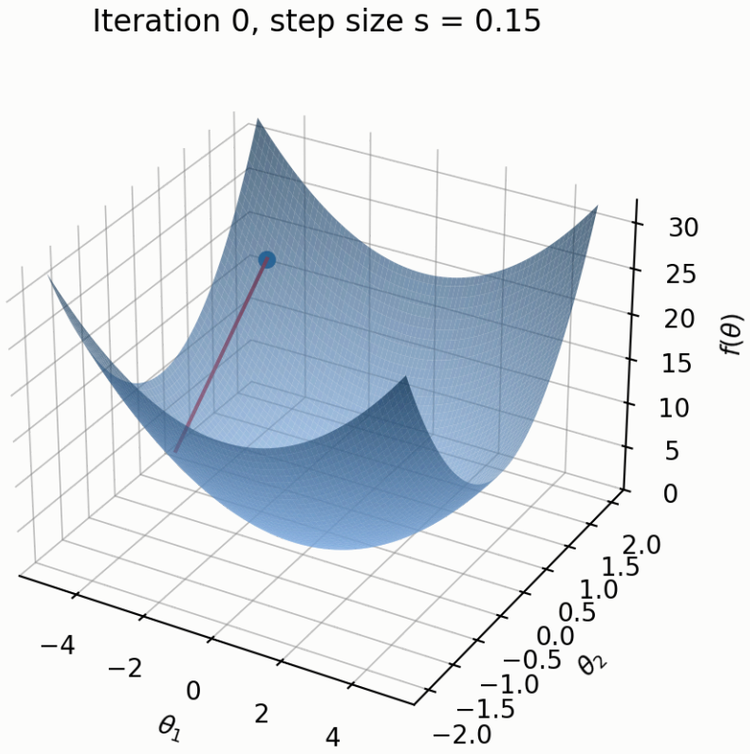
- Given $f(\theta) = \frac{1}{n}\sum_{j=1}^{n} f_j(\theta)$ to be minimized with respect to $\theta \in \mathbb{R}^p$ and step sizes s_i,

1. Initialize with some starting value $\theta^*_{(0)}$
2. Then, for each iteration, i,
    - Draw an index j_i uniformly at random from $\{1, \ldots, n\}$ and compute the gradient of that single term at $\theta^*_{(i)}$, i.e.,

$$\nabla f_{j_i}(\theta^*_{(i)}) = \left.\frac{\partial f_{j_i}(\theta)}{\partial \theta}\right|_{\theta=\theta^*_{(i)}}$$

    - Set $\theta^*_{(i+1)} = \theta^*_{(i)} - s_i \nabla f_{j_i}(\theta^*_{(i)})$
3. Repeat until convergence

Numerical Optimization IV



Backpropagation I

THE PROBLEM

- The numerical optimization, even in the SGD case, has incredibly high computational overhead
- Weights require the chain rule to compute derivatives

THE SOLUTION

- Backpropagation is a recursive application of the chain rule to compute the gradient of the loss
- Therefore, backpropagation eases the computational burden of training

Each SGD step uses a mini-batch B of observations (batch_size=32), so backpropagation computes the gradient of the batch-averaged loss rather than of a single R_i.

Backpropagation II

Recall the objective,

$$R(\theta) = \sum_{i=1}^{n} R_i(\theta) = \frac{1}{2} \sum_{i=1}^{n} (y_i - f_\theta(x_i))^2$$

A sum, which means the gradient is a sum over the n observations, so we need only consider one of these,

$$R_i(\theta) = \frac{1}{2} \left(y_i - \beta_0 - \sum_{k=1}^{K_1} \beta_k g_1 \left(\omega_{k0} + \sum_{j=1}^{p} \omega_{kj} x_{ij} \right) \right)^2$$

Backpropagation III

Let $z_{ik} = \omega_{k0} + \sum_{j=1}^{p} \omega_{kj} x_{ij} \Rightarrow f_\theta(x_i) = \beta_0 + \sum_{k=1}^{K_1} \beta_k g_1(z_{ik})$. Update β_k in one training step,

FORWARD PASS: COMPUTE THE FORECAST AND THE LOSS

$$\beta_k \rightarrow f_\theta(x_i) \rightarrow R_i(\theta)$$

BACKWARD PASS: MULTIPLY THE DERIVATIVES (CHAIN RULE)

$$\frac{\partial f_\theta(x_i)}{\partial \beta_k} = g_1(z_{ik}) \leftarrow \frac{\partial R_i(\theta)}{\partial f_\theta(x_i)} = -(y_i - f_\theta(x_i))$$

Computing the derivative w.r.t. β_k,

$$\begin{aligned} \frac{\partial R_i(\theta)}{\partial \beta_k} &= \frac{\partial R_i(\theta)}{\partial f_\theta(x_i)} \cdot \frac{\partial f_\theta(x_i)}{\partial \beta_k} \\ &= -(y_i - f_\theta(x_i)) \cdot g_1(z_{ik}) \end{aligned}$$

Backpropagation IV

Let $z_{ik} = \omega_{k0} + \sum_{j=1}^{p} \omega_{kj} x_{ij}$. Update ω_{kj} in one training step,

FORWARD PASS: COMPUTE THE FORECAST AND THE LOSS

$$\omega_{kj} \rightarrow z_{ik} \rightarrow g_1(z_{ik}) \rightarrow f_\theta(x_i) \rightarrow R_i(\theta)$$

BACKWARD PASS: MULTIPLY THE DERIVATIVES (CHAIN RULE)

$$\frac{\partial z_{ik}}{\partial \omega_{kj}} = x_{ij} \leftarrow \frac{\partial g_1(z_{ik})}{\partial z_{ik}} = g_1'(z_{ik}) \leftarrow \frac{\partial f_\theta(x_i)}{\partial g_1(z_{ik})} = \beta_k \leftarrow \frac{\partial R_i(\theta)}{\partial f_\theta(x_i)} = -(y_i - f_\theta(x_i))$$

Computing the derivative w.r.t. ω_{kj},

$$\begin{aligned} \frac{\partial R_i(\theta)}{\partial \omega_{kj}} &= \frac{\partial R_i(\theta)}{\partial f_\theta(x_i)} \cdot \frac{\partial f_\theta(x_i)}{\partial g_1(z_{ik})} \cdot \frac{\partial g_1(z_{ik})}{\partial z_{ik}} \cdot \frac{\partial z_{ik}}{\partial \omega_{kj}} \\ &= -(y_i - f_\theta(x_i)) \cdot \beta_k \cdot g_1'(z_{ik}) \cdot x_{ij} \end{aligned}$$

Backpropagation V: Update Rules

The two derivatives form the basis for the update rules, with step size s and one observation i per step

OUTPUT WEIGHT

$$\begin{aligned}\beta_k^{(t+1)} &= \beta_k^{(t)} - s\,\frac{\partial R_i}{\partial \beta_k} \\ &= \beta_k^{(t)} + s\,(y_i - f_\theta(x_i))\,g_1(z_{ik})\end{aligned}$$

HIDDEN WEIGHT

$$\begin{aligned}\omega_{kj}^{(t+1)} &= \omega_{kj}^{(t)} - s\,\frac{\partial R_i}{\partial \omega_{kj}} \\ &= \omega_{kj}^{(t)} + s\,(y_i - f_\theta(x_i))\,\beta_k\,g'(z_{ik})\,x_{ij}\end{aligned}$$

- Both updates contain the forecast error $(y_i - f_\theta(x_i))$
- This error term is attributed to the parameters via the terms $g(z_{ik})$ and $\beta_k g'(z_{ik})x_{ij}$

Optimization Algorithms

> Stochastic Gradient Descent is rarely used in practice today

BUILDING BLOCK

RMSProp

Uses smaller steps for parameters with large recent gradients

BUILDING BLOCK

Momentum

Averages past gradients to smooth noise and speed up

COMMONLY PREFERRED

ADAM

A combination of RMSProp and momentum (Kingma & Ba, 2014)

Specifying the Optimizer

Python Code

```
optimizer = torch.optim.Adam(model.parameters(), lr=0.001)
```

Common alternatives: `torch.optim.SGD`, `torch.optim.RMSprop`, `torch.optim.AdamW`

- `lr=0.001` is default for the `torch.optim.Adam` function.
- `model` is a name we specify for the neural network object, e.g., `model = NN(n_in=3)`

Guards Against Overfitting

Dropout

Randomly sets neurons to zero in each training step

ℓ_1 regularization

Penalizes the absolute value of the weights, favouring sparse networks

ℓ_2 regularization (weight decay)

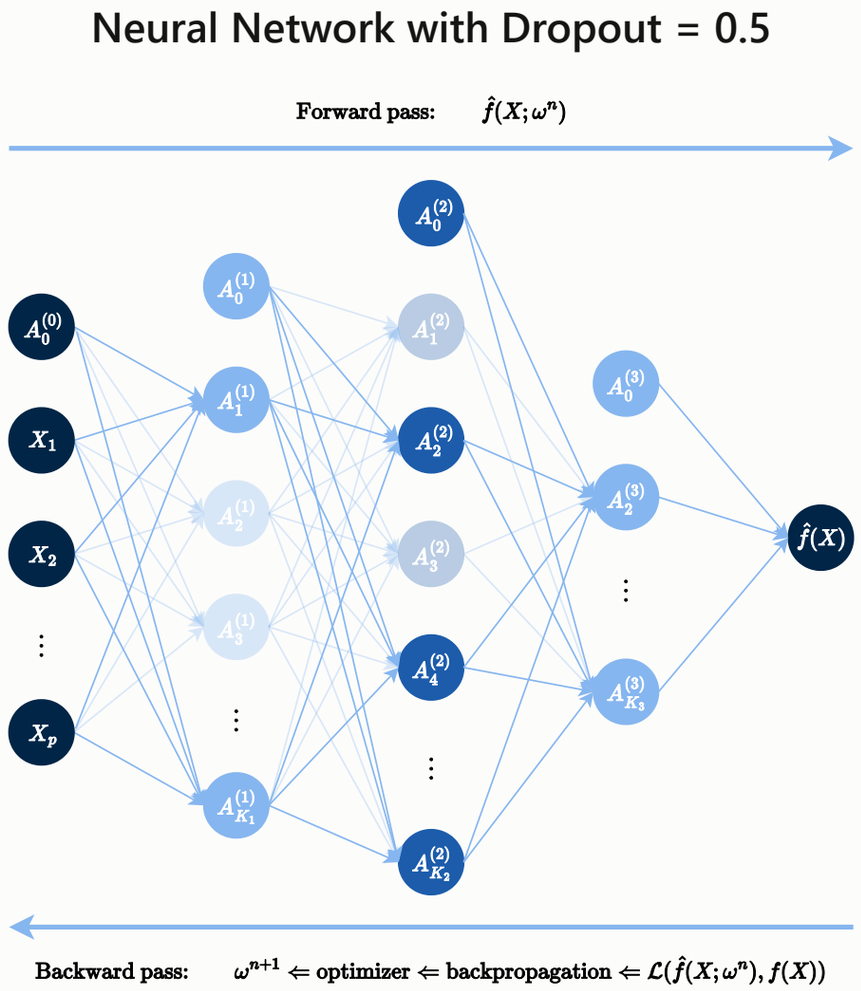
Shrinks the weights towards zero

Early stopping

Stops training when the validation loss no longer improves

Dropout

- Dropout (Srivastava et al., 2014): `nn.Dropout(p)` is applied in every training step
- Each neuron in the layer is set to zero with probability p, e.g., $p = 0.5$
- Neurons are made more independent, so the network behaves like an average of many thinned networks (think bagged trees)
- Dropout is not applied in evaluation

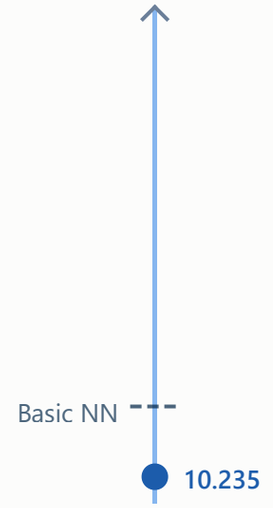


Dropout in PyTorch

Applying dropout to layer `fc1` with specified probability `p=0.5`

Python Code

```python
# in __init__
self.dropout = nn.Dropout(p=0.5)

# in forward
x = self.dropout(F.relu(self.fc1(x)))
```



ℓ_1 and ℓ_2 Regularization

ℓ_1: absolute value

Penalizes the absolute value of the weights, favouring sparse networks, with penalty added to the loss,

$$\mathcal{L}(\omega) + \lambda \sum_{h,k,j} \left| \omega_{kj}^{(h)} \right|$$

ℓ_2: weight decay

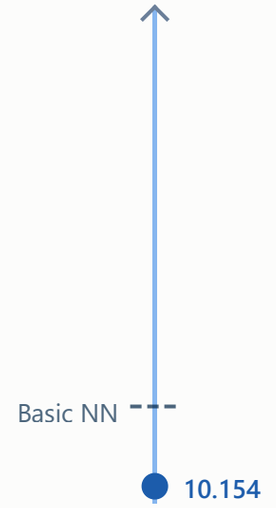
Weight decay (ℓ_2) adds a penalty to the loss, shrinking the weights towards zero,

$$\mathcal{L}(\omega) + \frac{\lambda}{2} \sum_{h,k,j} \left(\omega_{kj}^{(h)} \right)^2$$

ℓ_1 Regularization in PyTorch

Python Code

```python
l1_lambda = 0.0001

for inputs, targets in lTrain:
    ...
    loss = criterion(outputs, targets)
    loss = loss + l1_lambda * sum(param.abs().sum() for param in model.parameters())
```

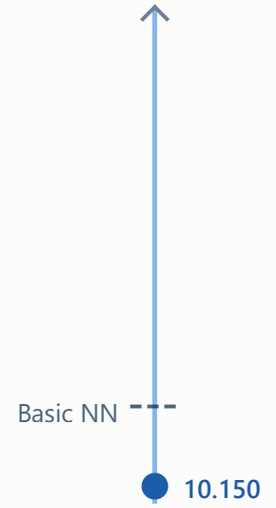


ℓ_2 Regularization in PyTorch

Weight decay is passed as an argument to the optimizer, e.g., `weight_decay=1e-4` in `Adam`,

Python Code

```python
optimizer = torch.optim.Adam(model.parameters(), lr=0.001, weight_decay=1e-4)
```



Up to you: Penalties

Compute $\sum |\omega|$ (the ℓ_1 term) and $\sum \omega^2$ (the ℓ_2 term) for the weights $\omega = (0.5, -1, 2)$.

Exercise | Start Over | Show Solution | Run Code

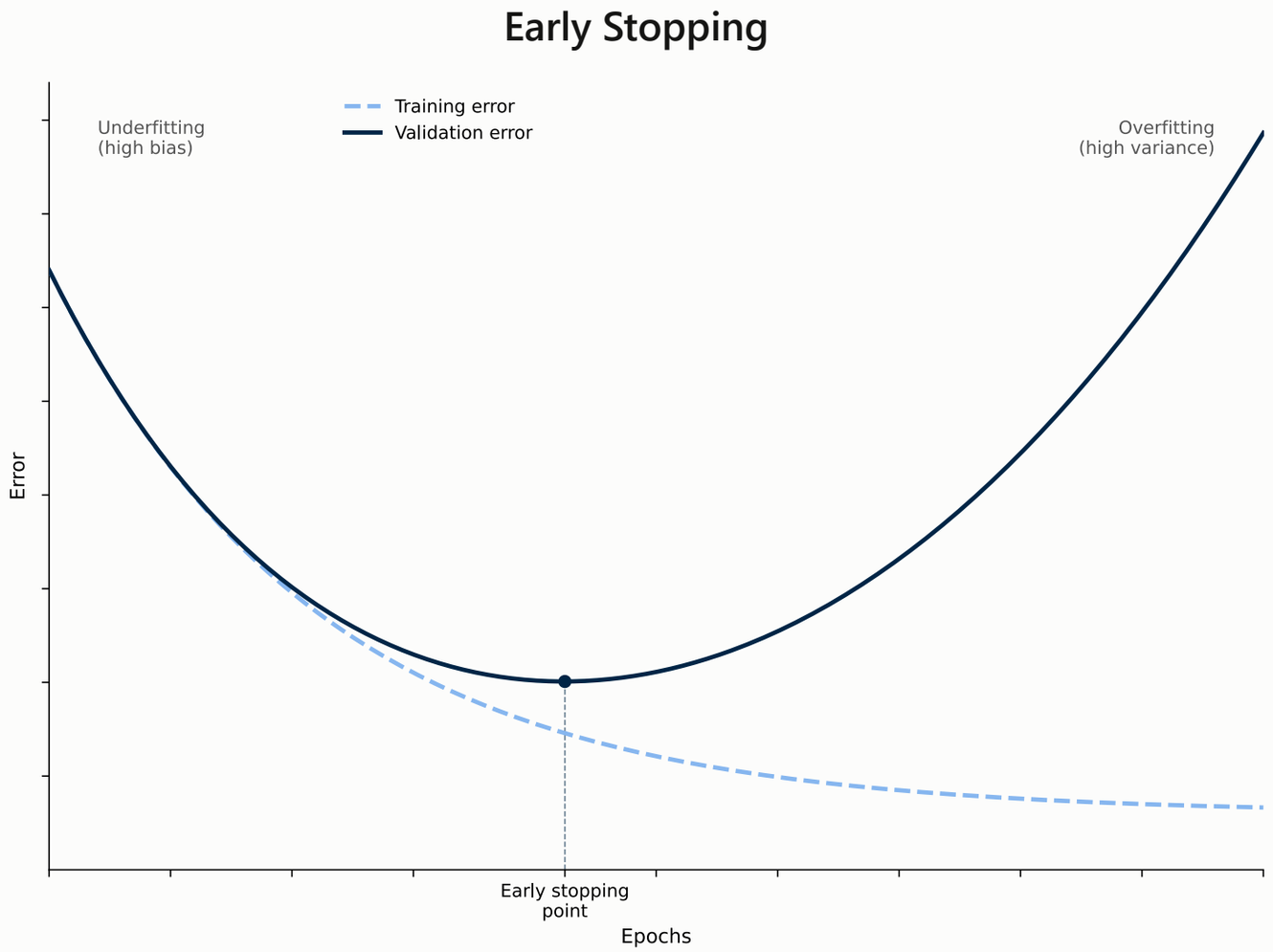
```
w = np.array([0.5, -1.0, 2.0])
l1 = ...
l2 = ...
print(f"L1 = {l1:.2f}, L2 = {l2:.2f}")
```

Epochs

> An epoch is a complete pass through the entire training dataset

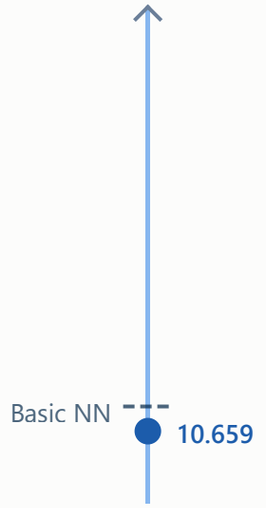
The training process is iterative and involves multiple **epochs**. The number of epochs can be 1) prespecified, 2) tuned, or 3) determined by early stopping

Early Stopping I



Early Stopping II

When the training accuracy keeps increasing, we continue training → The validation accuracy, our ability to generalize, starts to decrease after a certain point → No improvement for # `patience` epochs: stop and restore the best model state



Early stopping adds a few lines to the training loop. Refer to Lecture 4 Exercises

Loss Functions

We still consider only the regression task. Typical loss functions include,

`nn.MSELoss`

Squared error

`nn.L1Loss`

Absolute error

`nn.HuberLoss`

Squared for small errors, absolute for large (Huber, 1964) – remember for later

Example for the mean squared error,

Python Code

```python
criterion = nn.MSELoss()

for inputs, targets in lTrain:
    ...
    loss = criterion(outputs, targets)
```

We can write our own loss functions, but this is up to you if you wish to do so.

Training in PyTorch

Initializing Training

- The model is initialized and moved to the appropriate device (CPU or GPU)
- The loss function and optimizer are specified
- The number of epochs is set to 20

Python Code

```python
model = NN(n_in=X_train.shape[1]).to(device)

criterion = nn.MSELoss()
optimizer = torch.optim.Adam(model.parameters(), lr=0.001)
n_epochs = 20
```

One Training Step

1. `optimizer.zero_grad()`

   Reset the gradients

2. `model(inputs)`

   Forward pass: compute the forecasts

3. `criterion(outputs, targets)`

   Compute the loss

4. `loss.backward()`

   Backpropagation: compute the gradients

5. `optimizer.step()`

   Update the parameters

Main Training and Validation Loop I

Python Code

```python
for epoch in range(n_epochs):
    model.train()
    for inputs, targets in lTrain:
        inputs, targets = inputs.to(device), targets.to(device)

        optimizer.zero_grad()
        outputs = model(inputs)
        loss = criterion(outputs, targets)
        loss.backward()

        optimizer.step()
  ...
# Continued on next slide
```

Main Training and Validation Loop II

Python Code

```python
for epoch in range(n_epochs):
  ...
# Continued from previous slide
    model.eval()
    val_loss = 0.0
    with torch.no_grad():
        for inputs, targets in lVal:
            inputs, targets = inputs.to(device), targets.to(device)

            outputs = model(inputs)
            loss = criterion(outputs, targets)
            val_loss += loss.item() * inputs.size(0)

    val_loss /= len(lVal.dataset)
    print(f"Epoch {epoch + 1}: validation loss {val_loss:.4f}")
```

One Validation Step

1. `model.eval()`

   Switches to evaluation mode (leaving out dropout, batch normalization, etc.)

2. `torch.no_grad()`

   Disables gradient computations

3. `model(inputs)`, `criterion(outputs, targets)`

   Computes forecasts and loss

4. `val_loss += loss.item() * inputs.size(0)`

   Adds the batch's summed loss to `val_loss`

5. `val_loss /= len(lVal.dataset)`

   Divides `val_loss` by the number of validation observations to get the validation MSE

One Evaluation Pass

1. `model.eval()` and `torch.no_grad()`

Switches to evaluation mode (leaving out dropout, batch normalization, etc.) and disables gradient computation

2. `model(inputs)` over `lTest`

Computes the forecasts for the test sample

3. `y_scaler.inverse_transform`

Transforms the forecasts and the ground truth back to the original scale

4. Test MSE

Computes the mean squared error

5. OOS R^2 vs. training mean

Computes the out-of-sample R^2 relative to the training-mean forecast

Model Evaluation

Python Code

```python
model.eval()
lPred = []

with torch.no_grad():
    for inputs, _ in lTest:
        lPred.append(model(inputs.to(device)).cpu().numpy())

forecasts = np.concatenate(lPred)
forecasts_unstd = y_scaler.inverse_transform(forecasts)
y_unstd = y_scaler.inverse_transform(y_test.cpu())

mse = np.mean((y_unstd - forecasts_unstd) ** 2)
# Benchmark: training-sample mean forecast
y_bench = y_scaler.inverse_transform(y_train.cpu()).mean()
r2_oos = 1 - np.sum((y_unstd - forecasts_unstd) ** 2) / np.sum((y_unstd - y_bench) ** 2)
print(f"Test MSE: {mse:.4f} | Test OOS R2 (vs. train mean): {r2_oos:.3f}")
```

An Extended Pipeline (My Preferred Approach)

Reproducible

I set the seed for reproducibility

Tracked

I store the training and validation losses, time the run, and keep running loss book-keeping for each epoch to keep track of the training process

Monitored

I print these losses at the end of each epoch to monitor convergence

Limitations of Neural Networks

Jagged Intelligence

Models have incredible power, yet struggle with tasks like telling time (Stanford 2026 AI Index Report)

Black Box

Models are not interpretable with potentially harmful consequences

Data Hungry

Models require large amounts of data to train effectively, which may not be available in all domains

Computational overhead

Frequently requires GPUs, high memory, and long compute times

Beyond Feed-Forward Neural Networks

Broader Applications of Deep Learning

COMPUTER VISION

Convolutional Neural Networks

TIME SERIES MODELLING

Recurrent Neural Networks, LSTMs, Transformers

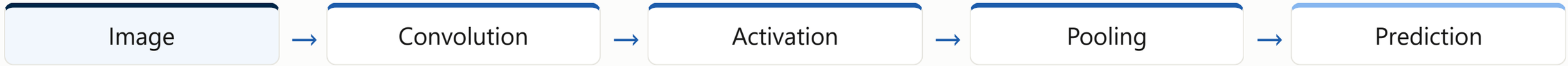
NATURAL LANGUAGE PROCESSING

Transformers (!) → Large Language Models

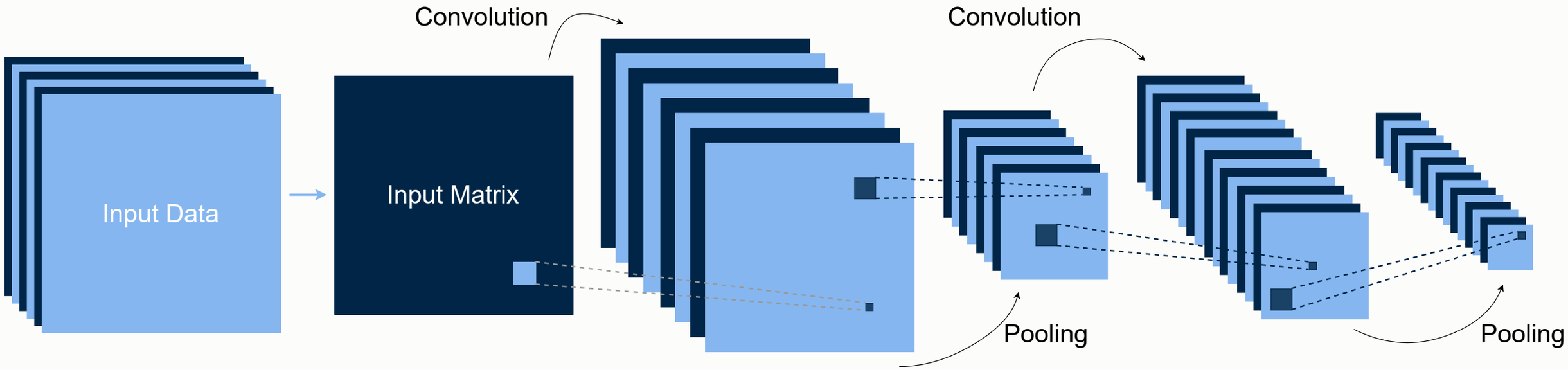
> Example: Computer vision for health care

Convolutional Neural Networks (CNNs)

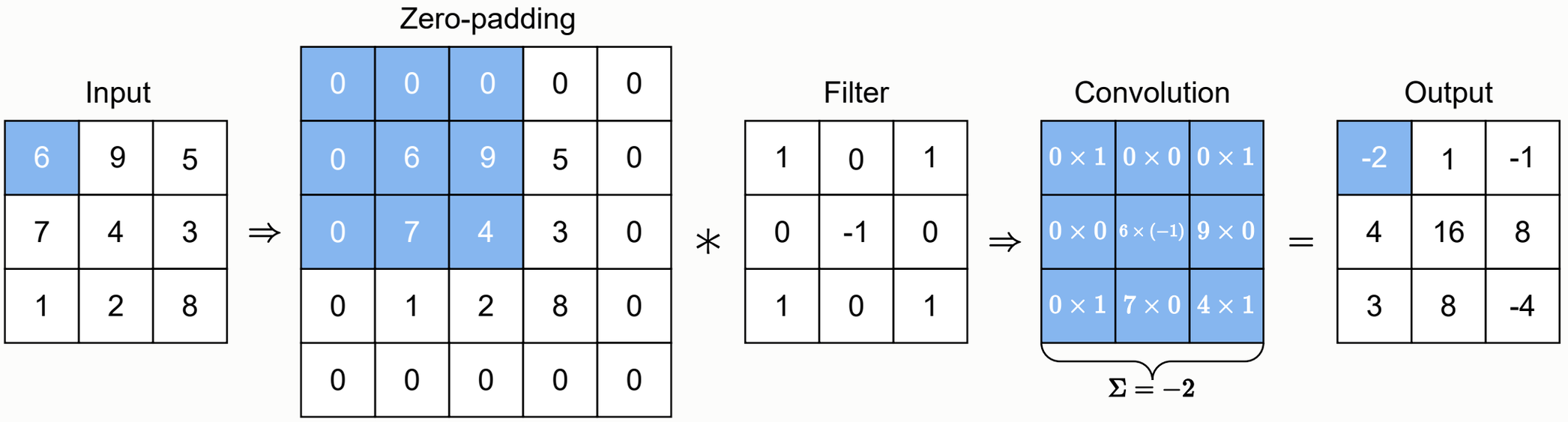
- Developed for computer vision tasks (LeCun et al., 1998)
- The classic deep learning architecture for image recognition and classification
- Pipeline consists of convolutional and pooling operations



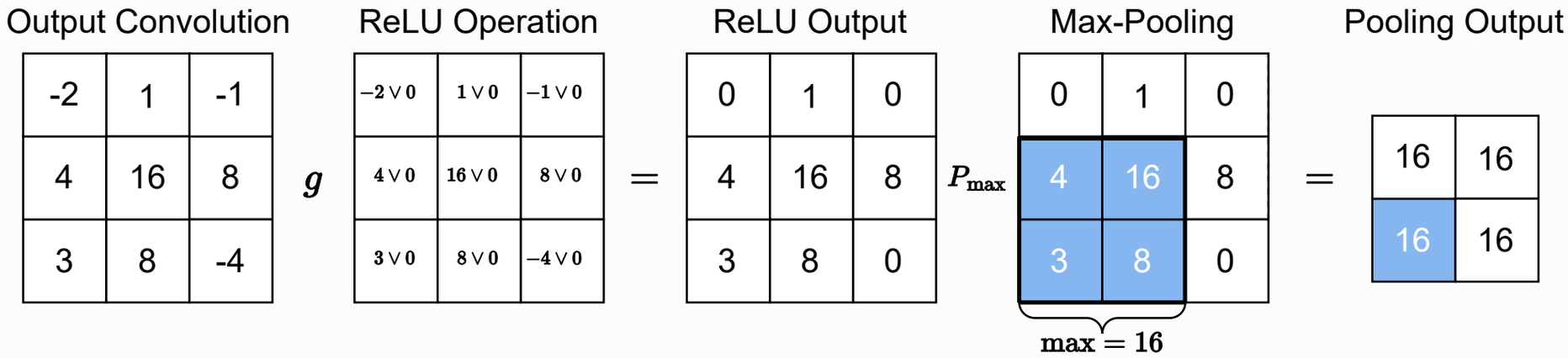
A Convolutional Neural Network Visualized



The Convolutional Operation



Applying Activations and Pooling



A Stylized Example: Handwritten Digits

Classic image dataset: 1,797 grayscale images of 8×8 pixels, showing the digits 0 to 9,

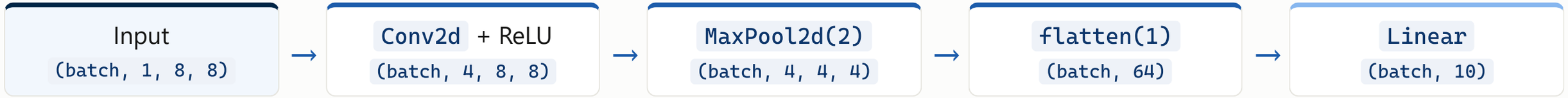
Python Code | Start Over | Run Code

```python
from sklearn.datasets import load_digits
digits = load_digits()

print(f"Label examples: {digits.target[0:20]}")
print(digits.images[0].astype(int))
```

```
Label examples: [0 1 2 3 4 5 6 7 8 9 0 1 2 3 4 5 6 7 8 9]
[[ 0  0  5 13  9  1  0  0]
 [ 0  0 13 15 10 15  5  0]
 [ 0  3 15  2  0 11  8  0]
 [ 0  4 12  0  0  8  8  0]
 [ 0  5  8  0  0  9  8  0]
 [ 0  4 11  0  1 12  7  0]
 [ 0  2 14  5 10 12  0  0]
 [ 0  0  6 13 10  0  0  0]]
```

The CNN input has shape `(batch, channels, height, width)`. A grayscale image has `channels` = 1

A Minimal CNN



Python Code

```python
class SimpleCNN(nn.Module):
    def __init__(self):
        super(SimpleCNN, self).__init__()
        self.conv = nn.Conv2d(1, 4, kernel_size=3, padding=1)  # 4 filters of size 3x3
        self.pool = nn.MaxPool2d(2)
        self.fc = nn.Linear(4 * 4 * 4, 10)                    # one score per digit

    def forward(self, x):                                     # x: (batch, 1, 8, 8)
        x = self.pool(F.relu(self.conv(x)))
        return self.fc(x.flatten(1))
```

Training the CNN

The task is classification into 10 classes, so the loss is chosen to be the **cross-entropy**,

Python Code

```python
model = SimpleCNN().to(device)
criterion = nn.CrossEntropyLoss()
optimizer = torch.optim.Adam(model.parameters(), lr=0.001)

# training loop: identical to the feedforward network

# evaluation: the predicted class is the one with the highest score
pred = model(X_test).argmax(dim=1)
accuracy = (pred == y_test).float().mean()
```

> Only the input shape, the layers and the loss change. The training loop stays the same

Forecasting from Images of Price Charts I

A more relevant application: Jiang, Kelly and Xiu (2023) turn the recent price history of stocks into images, then use a 2D CNN to forecast the direction of future returns

INPUT

A black-and-white image of the open-high-low-close bars, a moving average and volume bars of the last 5, 20 or 60 days.

All stocks on NYSE, AMEX, and NASDAQ

MODEL

The same ingredients as our example, only deeper: convolution, batch normalization, leaky ReLU and max-pooling, stacked in blocks.

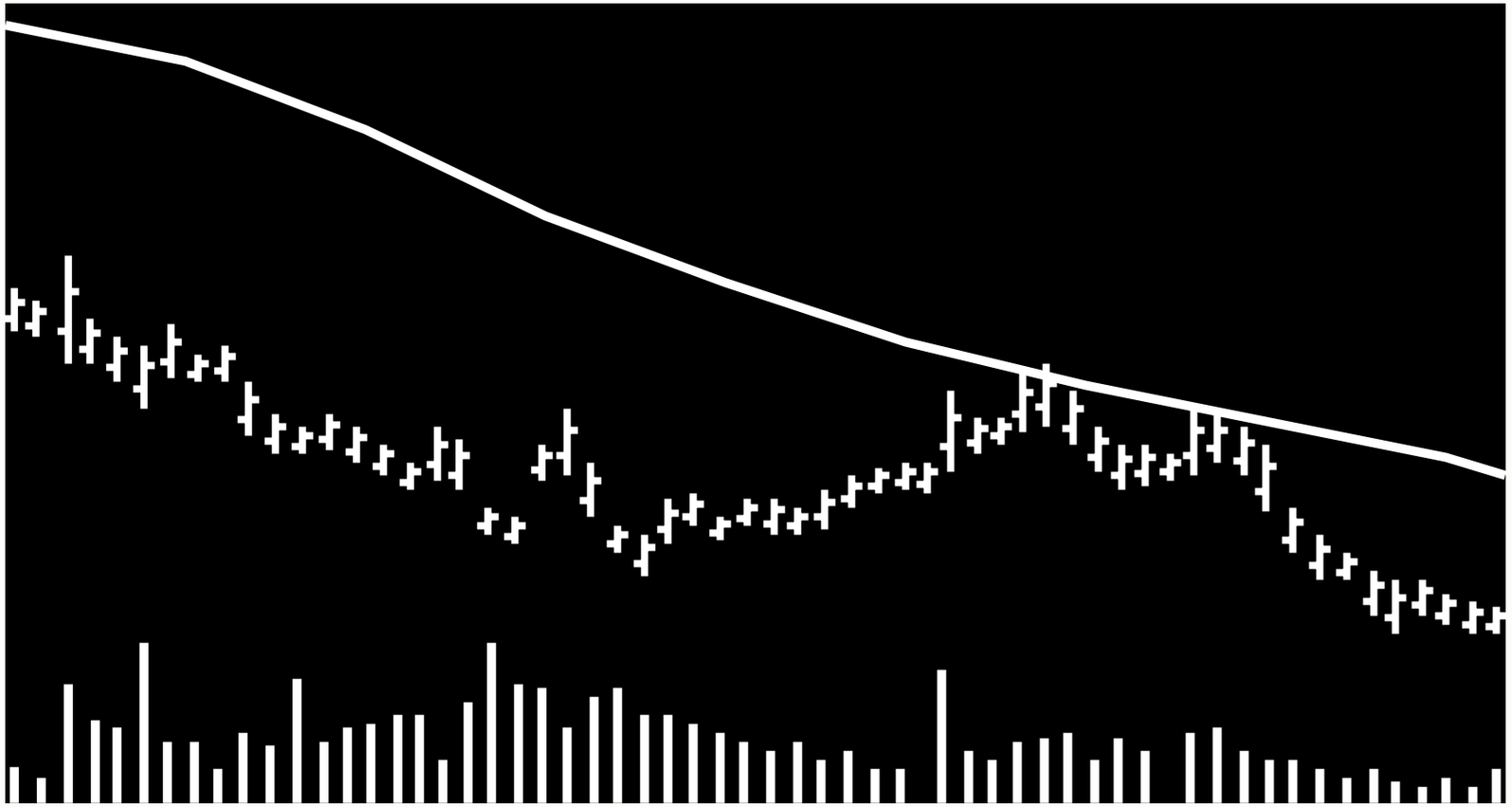
A fully connected layer with dropout and sigmoid activation produces the forecast

FORECAST

The probability that the subsequent return is positive

Forecasting from Images of Price Charts II

60 day OHLC image



From Jiang, Kelly and Xiu (2023), "(Re-)Imag(in)ing Price Trends". *Generated OHLC images with volume bar and moving average line. This figure displays market data image for 60 days of data.*

Recurrent Neural Networks (RNNs) and LSTMs

Feedforward neural networks (FNNs)

- Input: variables from $t-1$ (time series forecasting)
- Forecast: target in t
- No notion of order or history

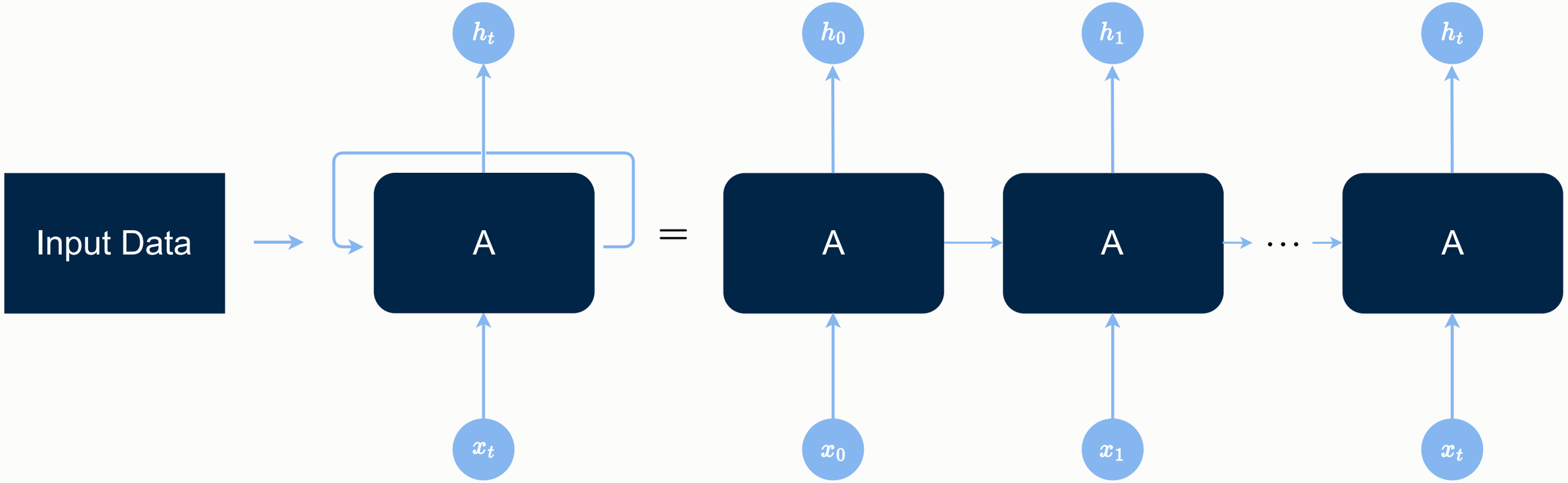
Recurrent neural networks (RNNs)

- Designed for sequential data, where the output at time t depends on previous inputs and outputs
- Sequential data common in time series and natural language processing

Long short-term memory networks (LSTMs)

- A type of RNN that can learn long-term dependencies
- Before 2017 (the transformer revolution, in a few slides), Google Translate used LSTMs
- Twin network: The GRU (Gated Recurrent Unit)

Recurrent Neural Networks Visualized



Forecasting Inflation from a Window of Quarters

We continue our inflation forecasting example, now in a recurrent setting, supporting time series consciousness

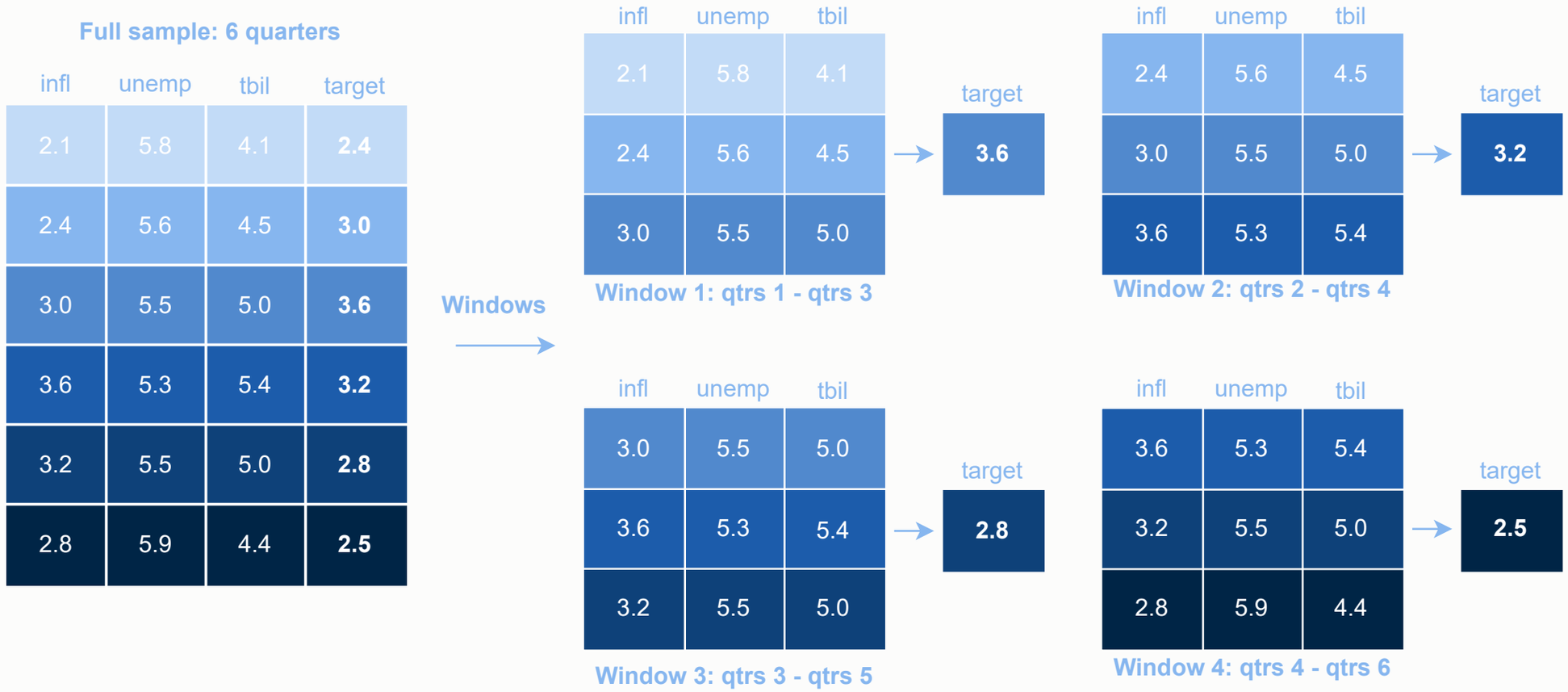
Feedforward network (so far)

- Input: the three variables of one quarter
- Forecast: inflation in the next quarter

Recurrent network

- Input: the last 8 quarters of inflation, unemployment and the T-bill rate
- Forecast: inflation in the next quarter

Windows of Quarters, Example



Windows have shape `(samples, quarters, variables)` = (4, 3, 3)

Target has shape `(samples, 1)` = (4, 1)

Setting up Windows I

`windows` has shape `(samples, quarters, variables)`,

Python Code

```python
qrts = 8                                        # quarters in a window
windows = np.lib.stride_tricks.sliding_window_view(X, qrts, axis=0)
windows = windows.transpose(0, 2, 1)            # (samples, quarters, variables)
y_win = target[qrts - 1:]                       # target of the window ending in t

n_train = len(y_train) - qrts + 1   # first qrts-1 quarters have no full window
n_val = len(y_val)                  # validation and test keep their size

Xseq_train, Xseq_tmp, yseq_train, yseq_tmp = train_test_split(
    windows, y_win, train_size=n_train, shuffle=False)
Xseq_val, Xseq_test, yseq_val, yseq_test = train_test_split(
    Xseq_tmp, yseq_tmp, train_size=n_val, shuffle=False)

# cont'd on next slide
```

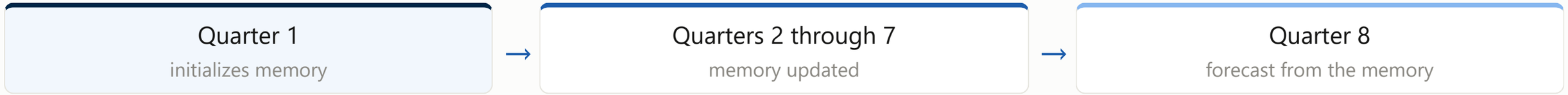
Setting up Windows II

Python Code

```python
# cont'd from previous slide

X_scaler = StandardScaler()
y_scaler = StandardScaler()

# The scaler expects 2D input, so reshape to (rows, 3) and back
Xseq_train = X_scaler.fit_transform(Xseq_train.reshape(-1, 3)).reshape(Xseq_train.shape)
Xseq_val = X_scaler.transform(Xseq_val.reshape(-1, 3)).reshape(Xseq_val.shape)
Xseq_test = X_scaler.transform(Xseq_test.reshape(-1, 3)).reshape(Xseq_test.shape)

yseq_train = y_scaler.fit_transform(yseq_train)
yseq_val = y_scaler.transform(yseq_val)
yseq_test = y_scaler.transform(yseq_test)
```

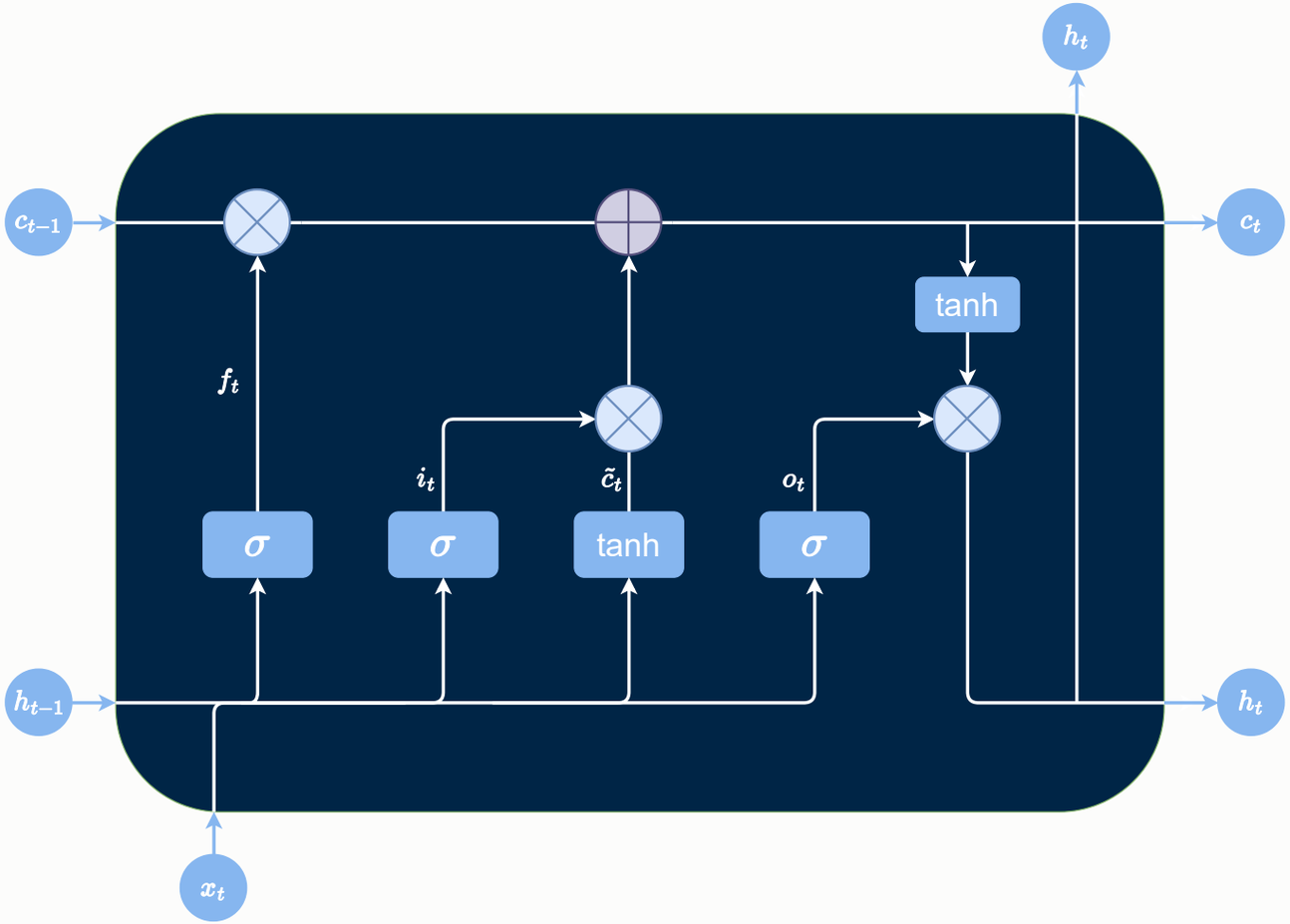
The Recurrent Idea

A recurrent network reads the window one quarter at a time, and keeps a memory of what it has seen: the **hidden state**,



- The same weights are used for every quarter
- The forecast is made from the memory after the last quarter
- The memory is the distinguishing characteristic of recurrent networks, LSTMs in particular

LSTM Unit Visualized



The LSTM Cell

The LSTM adds a cell state, a long-term memory, next to the hidden state. Gates between 0 (closed) and 1 (open) control it,

FORGET GATE

How much of the old memory is kept. Is last year's inflation spike still relevant?

INPUT GATE

How much of the new information is written to the memory. Is this quarter's jump in unemployment worth remembering?

OUTPUT GATE

How much of the memory is shown to the forecast. Which parts matter for next quarter?

A plain RNN rewrites its entire memory every quarter, so early quarters fade. The LSTM gates edit the cell state instead, so information persists $\rightarrow$ long-memory modeling feasible

A Recurrent Network in PyTorch

`qrts` indicates the number of quarters (in this case) in a window

Python Code

```python
class RecurrentNet(nn.Module):
    def __init__(self, n_features, hidden=16, cell="LSTM"):
        super(RecurrentNet, self).__init__()
        rnn = {"RNN": nn.RNN, "GRU": nn.GRU, "LSTM": nn.LSTM}[cell]
        self.rnn = rnn(n_features, hidden, batch_first=True)
        self.fc = nn.Linear(hidden, 1)

    def forward(self, x):                       # x: (batch, qrts, n_features)
        output, _ = self.rnn(x)                 # (batch, qrts, hidden)
        x = self.fc(output[:, -1, :])           # the last quarter's hidden state

        return x
```

The Transformer Revolution

Transformers (Vaswani et al., 2017) are the foundation of Large Language Models (LLMs)

ATTENTION

A mechanism that allows the model to focus on relevant parts of the input sequence when making predictions

SELF-ATTENTION

Transformers use self-attention mechanisms to process sequential data in parallel, allowing for better scalability and performance

Tip

The best intuitive introduction by far, in my opinion, is Section “The Transformer Architecture”, Chapter 15 of Deep Learning with Python (Chollet, 3rd edition, 2025).

Skip the code (which is geared towards TensorFlow) and focus on the intuition

Attention: The Idea

We stay with our inflation forecasting example.

An LSTM squeezes the window into one memory. Attention lets the forecast look at **every quarter directly** and decide how much each one matters.

QUERY: $\mathbf{Q}$

What information is this quarter looking for?

KEY: $\mathbf{K}$

What information does each quarter offer?

VALUE: $\mathbf{V}$

Information passed on

Maths to decorate the slide,

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^\intercal}{\sqrt{d}}\right)\mathbf{V}$$

Attention as a Lookup

Intuitively, attention works like a lookup/search engine:

Search engine		Attention
Search term	$\Leftrightarrow$	**Query**, $\mathbf{Q}$
Tags on the stored items	$\Leftrightarrow$	**Key**, $\mathbf{K}$
The stored items	$\Leftrightarrow$	**Value**, $\mathbf{V}$
Items returned, best match first	$\Leftrightarrow$	Weighted average of the values

- Here, stored items are the quarters of the window, and the search is run once every quarter
- The better the match, the larger the weight
- Weights are learned via backpropagation

A Self-Attention Network I

Self-attention: query, key and value all come from the same window. The forecast is made from the output of the last quarter,

Python Code

```python
class AttentionNet(nn.Module):
    def __init__(self, n_features, qrts, d=16):
        super(AttentionNet, self).__init__()
        self.embed = nn.Linear(n_features, d)           # quarter -> vector
        self.pos = nn.Parameter(torch.zeros(1, qrts, d))   # learned positions
        self.attn = nn.MultiheadAttention(d, num_heads=1, batch_first=True) # contains W_Q, W_K and W_V
        self.fc = nn.Linear(d, 1)

# cont'd on next slide
```

A Self-Attention Network II

Python Code

```python
# cont'd from previous slide
    def forward(self, x):                       # x: (batch, qrts, n_features)
        x = self.embed(x) + self.pos            # (batch, qrts, d)
        context, weights = self.attn(x, x, x)  # query = key = value = x
        x = self.fc(context[:, -1, :])          # the last quarter's context

        return x
```

`d` sets the size of the query, key and value vectors. A larger `d` stores more information per vector, but adds more parameters to learn

From Attention to a Transformer

Attention only recombines the quarters. We still need non-linear activations.

A Transformer block (Vaswani et al., 2017) adds them, and blocks are stacked,

Self-attention
information is mixed across quarters

→

Add and normalize
residual connection and layer norm

→

Feedforward network
two dense layers with ReLU, applied to each quarter

→

Add and normalize
residual connection and layer norm

THREE MORE INGREDIENTS

MULTI-HEAD

Several attention heads in parallel, so different patterns can be uncovered

POSITIONS

Attention is blind to order, so a position vector is added to each quarter (`self.pos` in the code)

CAUSAL MASK

A quarter may only attend to itself and earlier quarters → no look-ahead bias

LLMs and Generative AI

An LLM is a very large transformer with causal attention, trained to predict the next token. GPTs like e.g., ChatGPT abbreviates “Generative Pre-trained **Transformer**”

G
Generative (AI)

Produces new text one word at a time: predict, append, repeat (next slide)

P
Pre-trained

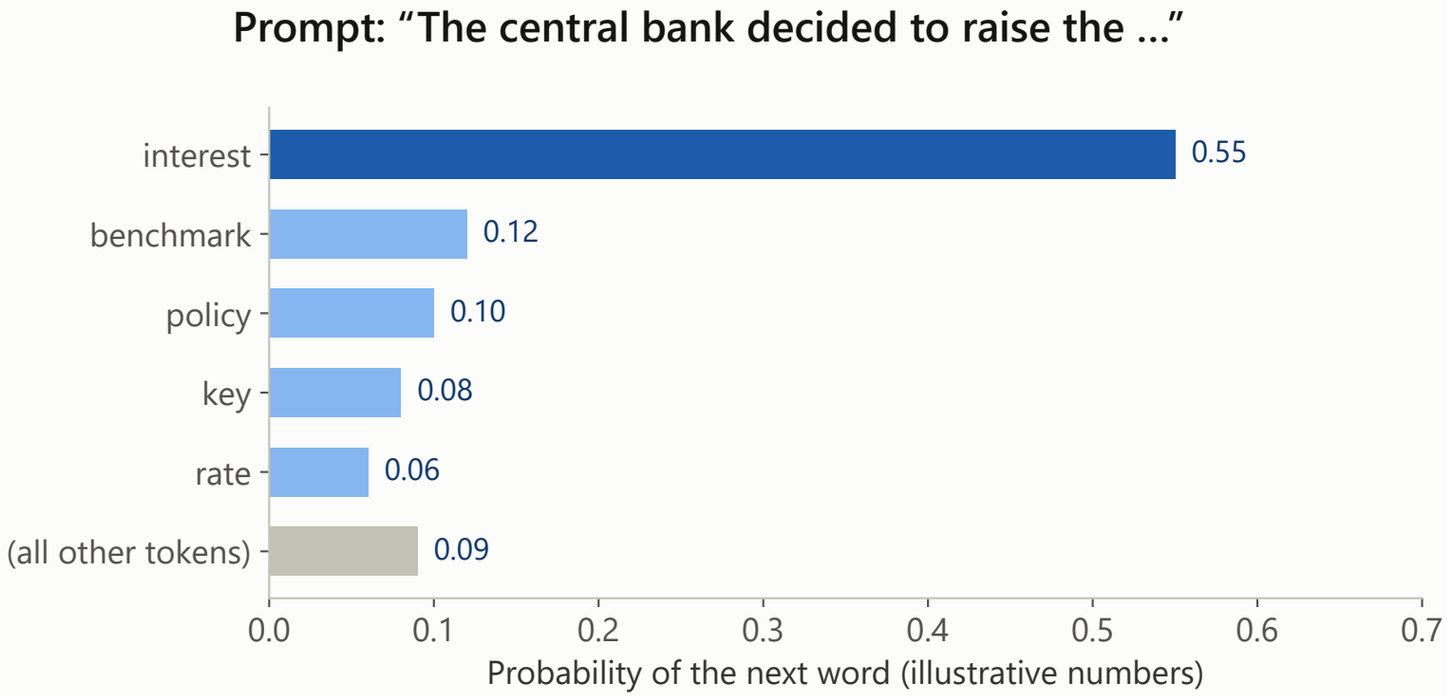
Trained on large corpora of text data to learn the statistical properties of language, by predicting the next word

T
Transformers

Stacked attention blocks with a causal mask, as on the previous slides

LLMs are Forecasting Models

When we chat with generative AI, the model **forecasts** the next word given all previous words, and we sample from this probability distribution,



The sampled word is appended and the model forecasts again: a recursive multi-step forecast

> The replies we receive are nothing but (incredibly good) predictions

Recent Deep Learning Breakthroughs

HARDWARE

The Chinese LLM LongCat-2.0 Model is trained without NVIDIA GPUs (Reuters, 2026)

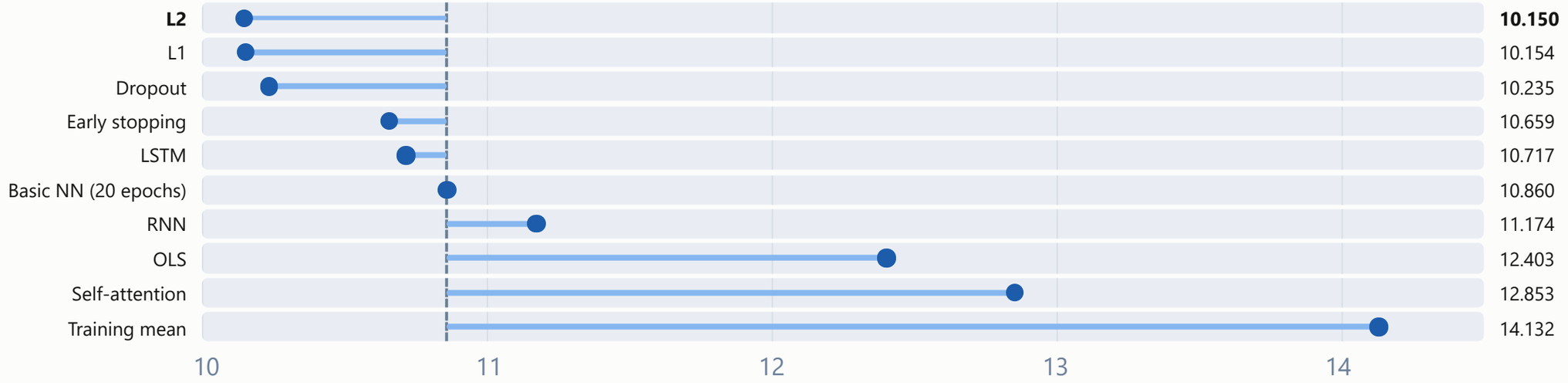
COMPUTER VISION MEETS MUSIC

MuSViT reads sheet music like computer vision models read images (Penarrubia et al., 2026)

MATHEMATICS

OpenAI and Anthropic (etc.) make progress in the field of mathematics

All Models Compared



Dashed line: basic NN.

References

Chollet, F. (2025). *Deep Learning with Python* (3rd ed.). Manning.

Huber, P. J. (1964). Robust estimation of a location parameter. *Annals of Mathematical Statistics*, 35(1), 73–101.

Huntr. (2026, 28 July). *Machine learning skills: What 10,704 job posts require (2026)*. https://huntr.co/resume-skills/machine-learning

Jiang, J., Kelly, B., & Xiu, D. (2023). (Re-)Imag(in)ing price trends. *Journal of Finance*, 78(6), 3193–3249.

Kingma, D. P., & Ba, J. (2014). Adam: A method for stochastic optimization. *arXiv:1412.6980*.

LeCun, Y., Bottou, L., Bengio, Y., & Haffner, P. (1998). Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, 86(11), 2278–2324.

Penarrubia, C., et al. (2026). MuSViT: A foundation vision model for sheet music representation. *ECCV*. https://grfia.dlsi.ua.es/musvit/

Reuters. (2026, 30 June). Meituan releases LongCat-2.0, trained without NVIDIA chips.

Second Talent. (2026, 21 September). *PyTorch vs. TensorFlow: Usage, popularity and performance*. https://www.secondtalent.com/resources/pytorch-vs-tensorflow-usage-popularity-and-performance/

Srivastava, N., Hinton, G., Krizhevsky, A., Sutskever, I., & Salakhutdinov, R. (2014). Dropout: A simple way to prevent neural networks from overfitting. *Journal of Machine Learning Research*, 15(56), 1929–1958.

Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., & Polosukhin, I. (2017). Attention is all you need. *NeurIPS*, 30.